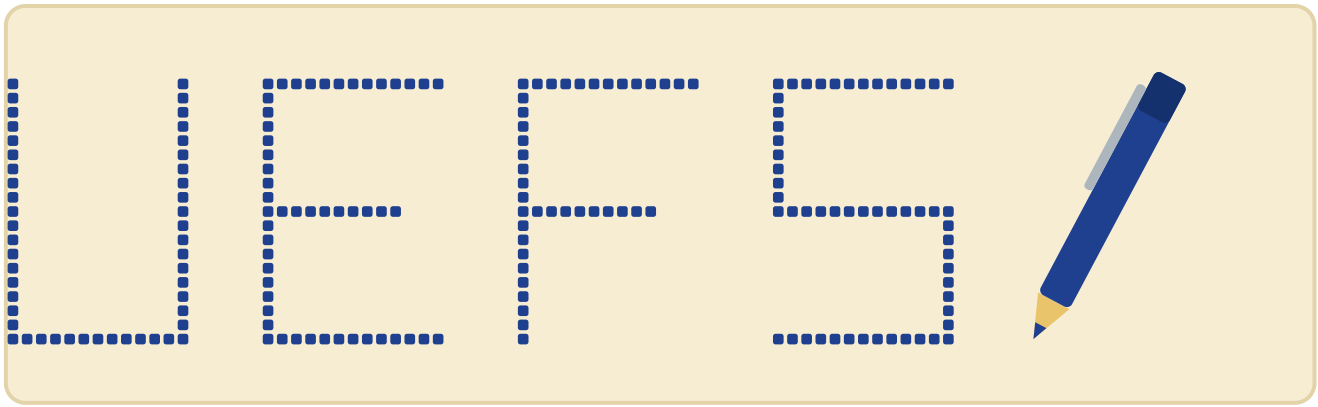
APOSTILA DO GRUPO · PROJETO DA IMPRESSORA DE ETIQUETAS

Impressora de Etiquetas CNC

Guia completo para quem está começando do zero: as peças, a montagem, a simulação no Wokwi e o código, explicados passo a passo.



Parte 1
O que é o projeto e como a máquina escreve

Parte 2
Cada peça: para que serve e como funciona

Parte 3
Montagem, primeiro teste e erros já resolvidos

Parte 4
A simulação e o código, linha por linha

Parte 5
Da simulação para a máquina real

Versão 1 · outubro de 2026

Inclui glossário e o código completo no apêndice

ANTES DE COMEÇAR

Como usar esta apostila

Esta apostila foi escrita para quem **nunca mexeu** com Arduino, motores ou eletrônica. Nada é dado como sabido: toda palavra técnica é explicada na primeira vez que aparece, e também está no **glossário**, no final.

O ideal é ler **na ordem**. Cada parte usa o que foi explicado na anterior:

Parte	O que você vai aprender
1. O projeto	O que é a impressora de etiquetas, quais são as partes da máquina e a ideia de como ela desenha uma letra.
2. As peças	O mínimo de eletricidade que você precisa e, peça por peça, o que cada componente faz: Arduino, motor de passo, driver A4988, Shield CNC, servo, fonte e capacitor.
3. Montagem e teste	Como o motor foi ligado na protoboard, a ordem segura de ligar as coisas, o código de teste explicado e todos os erros que já apareceram, com a solução de cada um.
4. A simulação	O que a simulação no Wokwi representa, como usar, e o código explicado bloco por bloco, com exemplos com números de verdade.
5. Próximos passos	O que muda quando sair do simulador para a máquina real: calibração, GRBL, G-code, Bluetooth, o roteiro de metas e o que falta comprar.

As caixas coloridas

ANALOGIA
Compara a ideia técnica com uma coisa do dia a dia.

POR QUÊ?
Explica o motivo de algo ser feito de um certo jeito.

CUIDADO
Coisas que podem queimar uma peça ou fazer você perder tempo. Leia sempre.

NA PRÁTICA
Dicas para quando vocês estiverem com as peças na mão.

EM RESUMO
O essencial da seção em poucas linhas. Bom para revisar.

Sobre o código

Os trechos de código aparecem em caixas cinza, com o **número da linha** à esquerda. Os números batem com o arquivo `sketch.ino` que vocês colaram no Wokwi, então dá para acompanhar com o arquivo aberto. O código completo está no Apêndice A.

Todos os desenhos de letras desta apostila foram gerados rodando o próprio código da simulação: o que aparece aqui é exatamente o que o Wokwi desenha.

Sumário

Parte 1 · Entendendo o projeto 5

1.1 O que vamos construir 5
1.2 Os quatro sistemas da máquina 6
1.3 Como a máquina desenha uma letra 7

Parte 2 · As peças, uma por uma 9

2.1 O mínimo de eletricidade que você precisa saber 9
2.2 Arduino Uno: o cérebro 10
2.3 Motor de passo NEMA17: o músculo preciso 11
2.4 Driver A4988: o tradutor de força 13
2.5 Ajustando a corrente do driver 14
2.6 Shield CNC v3: a placa de encaixe 15
2.7 Servo SG90: o braço da caneta 16
2.8 Fonte de 12 V e capacitor 17
2.9 Tela OLED (só na simulação) 18

Parte 3 · Montagem e primeiro teste 19

3.1 A ligação na protoboard 19
3.2 A ordem segura de ligar e desligar 20
3.3 O código de teste, linha por linha 20
3.4 Usando o PlatformIO no VS Code 23
3.5 Problemas que já apareceram (e como resolver) 24
3.6 Os códigos que vocês têm 25

Parte 4 · A simulação no Wokwi 27

4.1 O que é o Wokwi 27
4.2 O que cada peça da simulação representa 27
4.3 Como montar e rodar a simulação 28
4.4 O que você deve ver 28
4.5 Visão geral do código 29
4.6 Bibliotecas, pinos e ajustes 31
4.7 A memória da máquina: as variáveis de posição 32
4.8 setup() e loop() 33
4.9 escreverTexto(): da palavra para as letras 35
4.10 desenharLetra(): da letra para as retas 35
4.11 linha(): da grade para os passos 37
4.12 irPara(): andando em linha reta 38
4.13 Os passos, o pulso e a caneta 40
4.14 Acompanhando a letra L do começo ao fim 41
4.15 Exercício: criando letras novas 42

Parte 5 · Da simulação para a máquina de verdade 44

5.1 O que continua igual e o que muda 44
5.2 Calibração: quantos passos é um milímetro? 45
5.3 GRBL e G-code 46
5.4 Bluetooth e o aplicativo 47
5.5 Roteiro das próximas metas 48
5.6 O que ainda falta comprar 49

Consulta · Glossário 50

Consulta · Referências e links úteis 52

Apêndice A · O código completo da simulação 53

PARTE 1

Entendendo o projeto

Antes de falar de fios e código, é preciso entender o que a máquina faz e como ela consegue escrever uma letra. Esta parte não tem nada técnico: só a ideia geral.

1.1 O que vamos construir

O projeto é uma **máquina que escreve etiquetas com uma caneta comum**, num rolo de fita (fita crepe ou fita de etiqueta). Você digita uma palavra, por exemplo **UEFS**, e a máquina escreve essa palavra na fita, traço por traço.

Ela não funciona como uma impressora de papel, que solta tinta em milhares de pontinhos. Ela **desenha**, do jeito que uma pessoa faria: encosta a caneta, risca, levanta, vai para outro lugar, encosta de novo. Máquinas que desenham com caneta se chamam **plotters** (em português, "traçadores").

Existem plotters de mesa, que movem a caneta por cima de uma folha parada. O nosso é um **plotter de tambor** (em inglês, *drum plotter*): a superfície onde se escreve é um cilindro que gira. No nosso caso, o cilindro é o próprio rolo de fita.

> **ANALOGIA**
>
> Pense em escrever seu nome numa lata de refrigerante deitada. Com uma mão você gira a lata; com a outra, move a caneta para a esquerda e para a direita ao longo dela. Combinando os dois movimentos, dá para escrever qualquer coisa. A máquina faz exatamente isso: **um motor gira o rolo** e **outro motor move a caneta**.

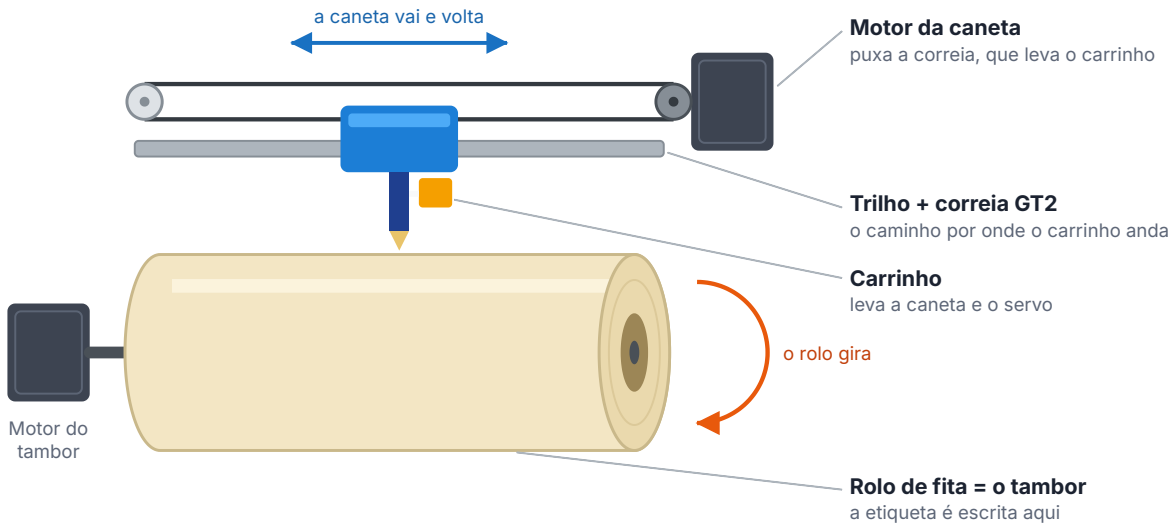


Figura 1 · A máquina vista de frente (desenho simplificado). Um motor gira o rolo de fita; o outro puxa uma correia que leva o carrinho com a caneta de um lado para o outro. O servo, no carrinho, encosta e afasta a caneta da fita.

Depois de escrita, a fita é desenrolada e vira uma etiqueta comum, com o texto na horizontal:

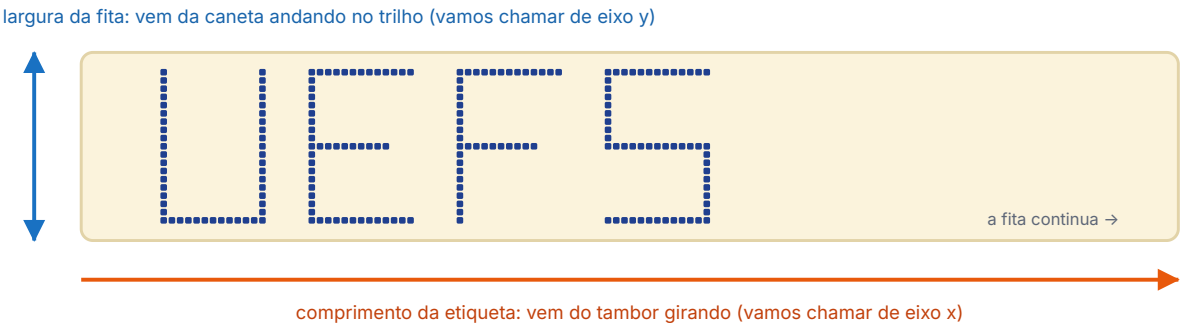


Figura 2 · A fita depois de desenrolada. O texto corre ao longo do comprimento da fita (o giro do tambor). A altura de cada letra fica na largura da fita (o movimento da caneta).

> **POR QUÊ?**
>
> Por que o texto fica "deitado" em volta do rolo? Porque a palavra é escrita ao longo do **comprimento** da fita, que é a direção em que o rolo gira. A **altura** das letras fica na largura da fita, que é a direção em que a caneta anda. Quando você desenrola a fita, o texto aparece normal.

1.2 Os quatro sistemas da máquina

O documento do projeto divide a máquina em quatro sistemas. Cada um tem uma tarefa:

Sistema	O que faz	Peças
Tambor	Gira o rolo de fita	Motor de passo NEMA17 (modelo 17HS2408), driver A4988, encaixe para o rolo
Eixo linear	Leva a caneta de um lado para o outro	Motor NEMA17, driver A4988, correia GT2, polia, trilho (guia linear)
Caneta	Encosta e afasta a caneta da fita	Mini servo motor SG90
Controle	O "cérebro": recebe o texto e comanda todo o resto	Arduino Uno R3 + Shield CNC v3 (e, mais para frente, um módulo Bluetooth HC-05)

Além deles, tem a **alimentação**: uma fonte de 12 V que dá energia aos motores e um capacitor de 100 µF que protege os drivers. Tudo isso é explicado peça por peça na Parte 2.

O documento do projeto cita duas opções para o eixo linear: correia com trilho (modelo 1) ou o mecanismo de um leitor de DVD (modelo 2). O grupo escolheu o **modelo 1, com correia**.

A Figura 3 mostra como as peças "conversam". O texto sai do computador (ou, no futuro, do celular), chega no Arduino, e o Arduino decide cada movimento. Ele manda ordens para os drivers e para o servo. Os drivers pegam energia da fonte e entregam aos motores, na hora certa.

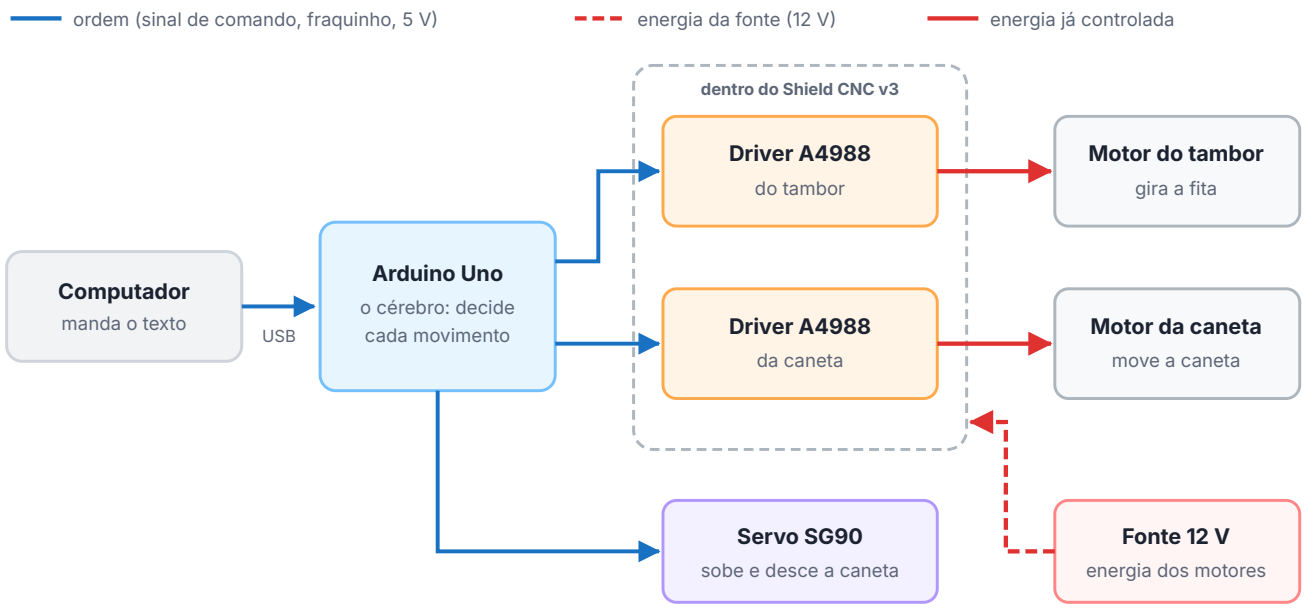


Figura 3 · Quem manda em quem. O Arduino só dá ordens (setas azuis). Quem entrega força aos motores são os drivers, usando a energia da fonte (vermelho).

> **ANALOGIA**
>
> O Arduino é como o **maestro** de uma orquestra: ele não toca nenhum instrumento, mas diz a cada músico o que fazer e quando. Os drivers são os músicos, e os motores são os instrumentos. A fonte é a energia que os músicos usam para tocar.

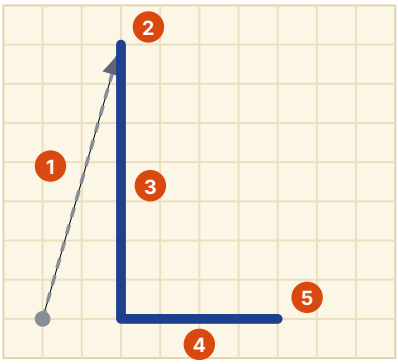
1.3 Como a máquina desenha uma letra

Imagine a fita como uma folha **quadriculada**. Qualquer ponto dela pode ser descrito por dois números:

- **x**: quanto o tambor girou, ou seja, em que posição do **comprimento** da fita a caneta está;
- **y**: onde a caneta está no trilho, ou seja, em que posição da **largura** da fita.

Esse par de números, escrito como **(x, y)**, se chama **coordenada**. É a mesma ideia da batalha naval: "B-4" diz exatamente qual quadradinho você quer atingir. Aqui, "(12, 30)" diz exatamente onde a caneta deve estar.

Para desenhar uma reta, a máquina sempre faz a mesma coisa: leva a caneta, **levantada**, até o começo da reta; **abaixa** a caneta; move os motores até o fim da reta; e **levanta** a caneta de novo. Veja a letra L:



1. Caneta **levantada**: os motores levam a caneta até o topo do L.
2. O servo **abaixa** a caneta. Ela encosta na fita.
3. Só o motor da caneta anda: sai o traço vertical.
4. Só o tambor gira: sai o traço horizontal.
5. O servo **levanta** a caneta. Letra pronta!

Figura 4 · Desenhando um L. A linha tracejada é a caneta se movendo levantada (não risca nada). As linhas azuis são os traços de verdade.

E as retas inclinadas, como as do V ou do A? Nesse caso **os dois motores andam ao mesmo tempo**, cada um na sua proporção. Isso está explicado em detalhes na seção 4.12.

EM RESUMO

A máquina só sabe fazer **três coisas**: girar o tambor, mover a caneta e subir/descer a caneta. Toda letra é uma sequência dessas três ações. O trabalho do código é transformar uma palavra digitada nessa sequência de movimentos.

PARTE 2

As peças, uma por uma

Aqui cada componente é apresentado com calma: o que é, para que serve no projeto e os cuidados que ele exige. Antes, uma seção curta com as poucas ideias de eletricidade que aparecem o tempo todo.

2.1 O mínimo de eletricidade que você precisa saber

Você não precisa saber eletrônica para montar a impressora, mas cinco ideias aparecem o tempo todo. Uma comparação com água ajuda bastante:

Ideia	Unidade	Na mangueira de água	No projeto
Tensão	volt (V)	A pressão da água	5 V no Arduino; 12 V na fonte dos motores
Corrente	ampere (A)	Quantos litros passam por segundo	Um pino do Arduino aguenta uns 0,02 A (20 mA); cada bobina do motor usa até 0,6 A
Resistência	ohm (Ω)	O quanto a mangueira é estreita	Cada bobina do motor tem 8 Ω

Repare na diferença: o motor precisa de uma corrente **30 vezes maior** do que um pino do Arduino consegue dar. Guarde isso: é o motivo de existir o driver (seção 2.4).

GND: o "chão" da eletricidade

GND (do inglês *ground*, "terra") é o ponto de 0 V. Toda tensão é medida em relação a ele, do mesmo jeito que a altura de um prédio é medida a partir do chão. Os pinos marcados GND, "−" ou "terra" são todos esse ponto de referência.

> **REGRA DE OURO: GND COMUM**
>
> Peças que conversam entre si precisam ter os GND **ligados juntos**. Se o Arduino e a fonte dos motores tiverem "chãos" diferentes, o "5 V" de um não significa nada para o outro, e os sinais viram bagunça. É como duas pessoas medindo alturas a partir de andares diferentes de um prédio.

HIGH e LOW: a "língua" do Arduino

Os pinos digitais do Arduino só têm dois estados, como um interruptor: **HIGH** (5 V, "ligado", 1) e **LOW** (0 V, "desligado", 0). Toda a conversa entre o Arduino e os drivers é feita ligando e desligando pinos, muito rápido.

Os dois "mundos": comando e potência

No projeto existem dois mundos que não podem se misturar:

- **Comando** (ou lógica): 5 V e correntes minúsculas. É o mundo do Arduino e dos sinais que ele manda.

- **Potência**: 12 V e correntes maiores. É o mundo da fonte e das bobinas dos motores.

O driver A4988 é a ponte entre os dois: recebe ordens do mundo de comando e entrega energia no mundo de potência.

> **CUIDADO**
> Nunca ligue os 12 V da fonte num pino do Arduino, no pino 5V ou no pino VDD do driver. Isso queima o Arduino ou o driver na hora. Os 12 V só vão no **VMOT** do driver (ou no borne de energia do Shield CNC).

2.2 Arduino Uno: o cérebro

O Arduino Uno é um **computador minúsculo** numa placa. O "computador" de verdade é o chip **ATmega328P**. Ele roda um único programa, que vocês gravam pelo cabo USB, e fica repetindo esse programa enquanto estiver ligado.

- **Velocidade:** 16 MHz, ou seja, 16 milhões de operações simples por segundo.
- **Memória de programa (flash):** 32 KB. É onde o código fica guardado, mesmo com o Arduino desligado. O código de teste do motor usou só 6,7% dela.
- **Memória de trabalho (RAM):** 2 KB. É onde ficam as variáveis enquanto o programa roda. Isso é pouquíssimo: um celular tem milhões de vezes mais. Por isso o código precisa ser enxuto. Só a tela da simulação ocupa metade dessa RAM (1 KB, um bit para cada um dos 128 × 64 pontos).

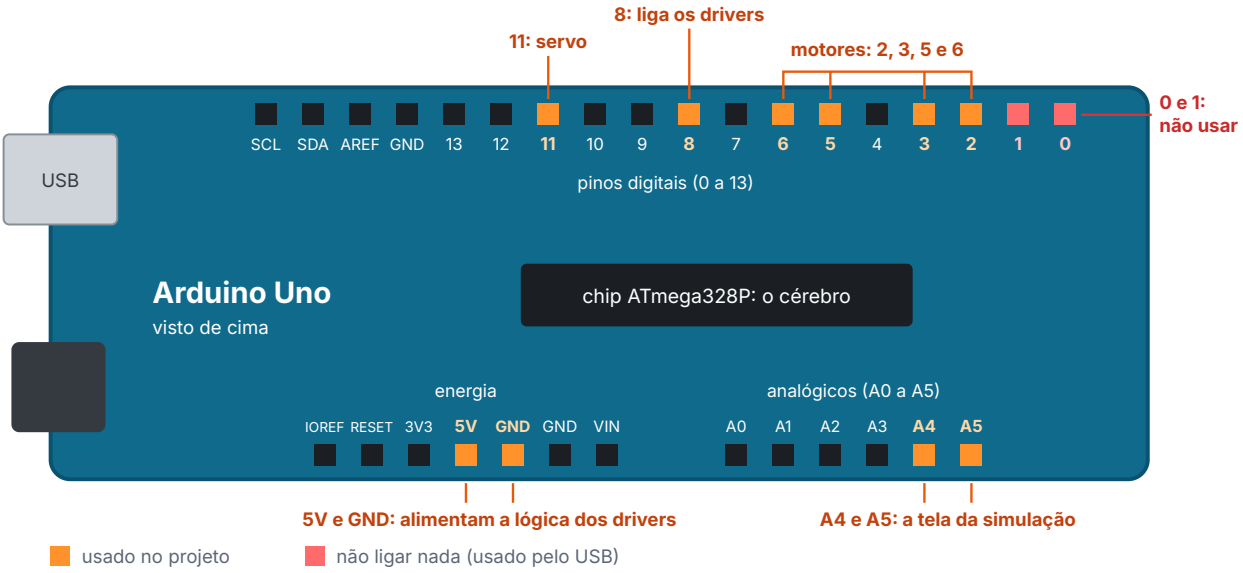


Figura 5 · Os pinos do Arduino Uno que o projeto usa (com o Shield CNC e o servo, como na simulação). Os pinos 0 e 1 são o caminho do USB: qualquer fio neles atrapalha a gravação do código.

A placa de vocês: um Uno compatível com chip CH340

O Uno de vocês é uma versão compatível (um "clone"). Ele usa um chip chamado **CH340** para conversar pelo USB, e por isso aparece no Windows como `USB-SERIAL CH340 (COM3)`. **COM3** é o nome da porta: é como o "endereço" onde o computador encontra o Arduino. Em outro computador, o número pode ser diferente.

Como é um programa de Arduino

Todo programa de Arduino tem duas partes obrigatórias:

- `setup()`: roda **uma vez só**, logo que o Arduino liga. Serve para preparar as coisas.
- `loop()`: roda **de novo e de novo**, sem parar, até desligar.

> **ANALOGIA**
> Numa padaria, o `setup()` é abrir as portas e ligar o forno de manhã (uma vez). O `loop()` é atender os clientes: chega um, atende, chega outro, atende, o dia inteiro.

> **NA PRÁTICA**
> **Gravar** (ou fazer *upload*) é mandar o programa do computador para o Arduino. Depois de gravado, o programa fica guardado no chip mesmo sem energia. Na próxima vez que o Arduino ligar, ele roda o mesmo programa de novo, sozinho.

2.3 Motor de passo NEMA17: o músculo preciso

Um motor comum, de ventilador ou de carrinho, gira sem parar enquanto recebe energia. Ele não sabe onde está e não consegue parar num ângulo exato.

Um **motor de passo** é diferente: ele gira em pequenos saltos de tamanho fixo, chamados **passos**. Cada ordem que ele recebe vale um passo. Pense no ponteiro dos segundos de um relógio de parede, que anda de "tique" em "tique".

Os motores de vocês andam **1,8° por passo**. Como uma volta tem 360°, uma volta completa tem **360 ÷ 1,8 = 200 passos**.

> **POR QUE ISSO É TÃO ÚTIL?**
> Se o Arduino mandar 50 pulsos, o motor gira exatamente 1/4 de volta (50 ÷ 200). Contando os pulsos que mandou, o Arduino **sabe onde o motor está**, sem precisar de nenhum sensor. É por isso que impressoras 3D, máquinas CNC e a nossa impressora usam motor de passo.

Como ele funciona por dentro

Dentro do motor há um **ímã** que pode girar (o rotor) e, em volta dele, **eletroímãs** fixos chamados **bobinas**. Quando passa corrente numa bobina, ela vira um ímã e puxa o rotor. Ligando as bobinas numa sequência certa, o rotor vai "seguindo", passo a passo:

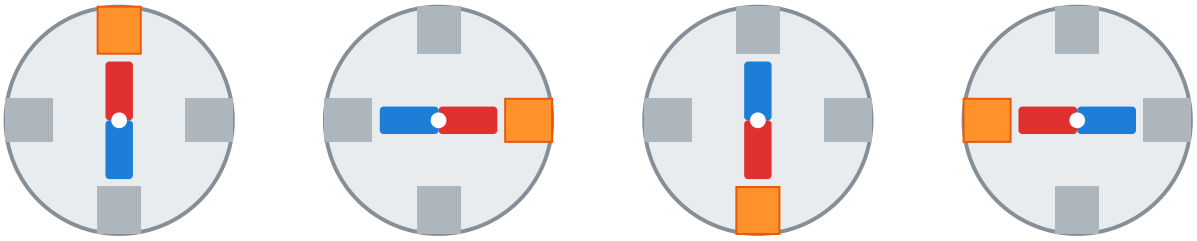


Figura 6 · Um motor de passo por dentro (bem simplificado). Laranja: o polo da bobina que está ligado. Vermelho: o polo norte do ímã, que é puxado. Aqui cada pulso gira 90°; no motor de verdade há dezenas de "dentinhos" e cada passo é de só 1,8°, mas a ideia é a mesma.

> **CUIDADO: "PERDER PASSOS"**
>
> Se o motor for forçado além do que aguenta (rápido demais, com peso demais ou com pouca corrente), ele **perde passos**: o Arduino acha que ele andou, mas ele não andou. Como ninguém confere, o erro se acumula e as letras saem tortas. Por isso a velocidade não pode ser alta demais.

Quatro fios, duas bobinas

O NEMA17 de vocês tem **4 fios**: dois para cada bobina, chamadas **A** e **B**. Para ligar no driver, é preciso saber quais dois fios são da mesma bobina (um "par"). Dois jeitos de descobrir:

- **Com multímetro** (na escala de resistência, Ω): entre dois fios do mesmo par aparece um valor baixo, cerca de **8 Ω** no 17HS2408. Entre fios de pares diferentes não há ligação (o multímetro mostra "OL" ou "1").
- **Sem multímetro**: com o motor desligado de tudo, gire o eixo com a mão (gira fácil). Encoste dois fios um no outro e gire de novo. Se ficou **mais duro**, esses dois são um par.

Com o driver ligado e o motor parado, o eixo fica "travado", difícil de girar com a mão. Isso é normal: o driver mantém uma bobina ligada para **segurar a posição**.

A ficha do motor de vocês

NEMA17 não é o modelo, é só o tamanho da frente do motor: 42 × 42 mm (1,7 polegada). O modelo de vocês é o **17HS2408**. Dados do catálogo do fabricante:

Ângulo por passo	**1,8°** (200 passos por volta)
Corrente por fase (por bobina)	**0,6 A**
Resistência por fase	**8 Ω**
Comprimento do corpo	28 mm (é um NEMA17 "baixinho")
Torque de retenção	12 N·cm (pequeno, mas suficiente para um rolo de fita e uma caneta)
Fios	4 (bipolar)

2.4 Driver A4988: o tradutor de força

O Arduino não consegue ligar o motor sozinho, por dois motivos:

1. **Força:** um pino do Arduino aguenta uns 20 mA. Cada bobina do motor quer até 600 mA, e a 12 V.
2. **Trabalho:** para girar o motor é preciso ligar as bobinas numa sequência certa, invertendo o sentido da corrente nas horas certas (a Figura 6). Fazer isso com o Arduino daria muito trabalho.

O **driver A4988** resolve as duas coisas. E o melhor: o Arduino só precisa de **dois fios** para comandá-lo, o **STEP** (passo) e o **DIR** (direção).

> **ANALOGIA**
>
> O driver é como o **pedal do acelerador** de um carro. O pé faz pouquíssima força, mas o pedal libera muita potência do motor. O Arduino "pisa no pedal" com sinais fraquinhos de 5 V; o driver libera a energia forte da fonte para as bobinas.

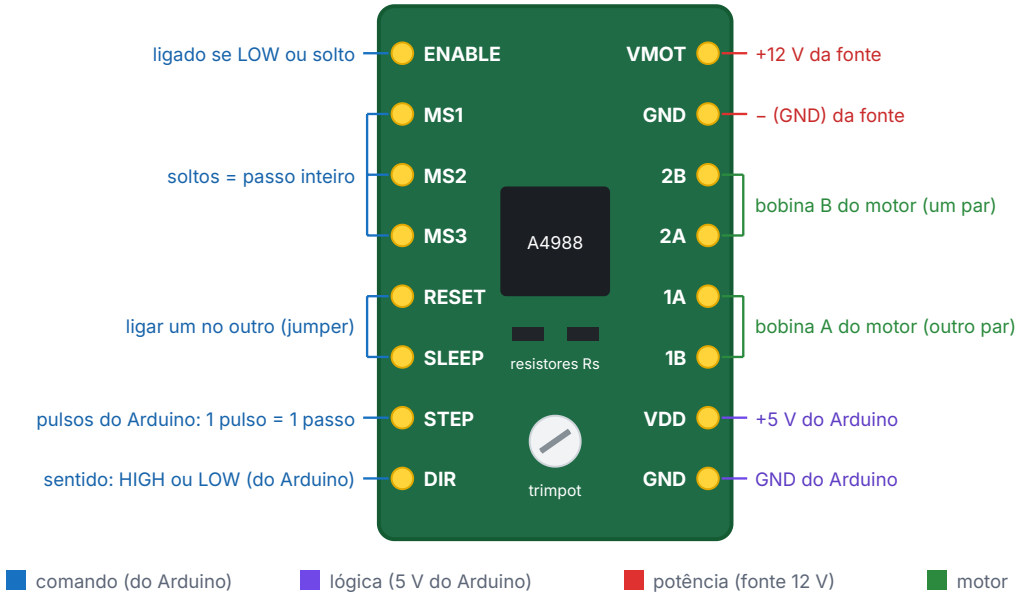


Figura 7 · Os 16 pinos do A4988 e para onde cada um vai. Esta é a ordem do módulo padrão (Pololu e a maioria dos clones), visto de cima com o trimpot embaixo. Antes de ligar, confira com o texto impresso no seu módulo.

Pino	Para que serve	Onde liga no projeto
`STEP`	Cada pulso (LOW → HIGH) faz o motor andar 1 passo	Pino do Arduino (3 no teste da protoboard)
`DIR`	HIGH = gira para um lado; LOW = para o outro	Pino do Arduino (2 no teste da protoboard)
`ENABLE`	LOW = driver ligado; HIGH = desligado (motor fica solto)	Solto no teste (o driver fica ligado); pino 8 no Shield CNC
`SLEEP, RESET`	Precisam estar em HIGH para o driver funcionar	Ligados um no outro (jumper)
`MS1, MS2, MS3`	Escolhem o micropasso (tabela abaixo)	Soltos = passo inteiro
`VDD, GND`	Alimentam o "cérebro" do driver (3 a 5,5 V)	5V e GND do Arduino
`VMOT, GND`	Energia para os motores (8 a 35 V)	+ e – da fonte de 12 V
`1A, 1B`	Bobina A do motor	Um par de fios do motor

Pino	Para que serve	Onde liga no projeto
`2A, 2B`	Bobina B do motor	O outro par

Micropasso (microstepping)

O A4988 consegue dividir cada passo em pedaços menores, deixando o movimento mais suave e mais preciso. Quem escolhe são os pinos MS1, MS2 e MS3:

MS1	MS2	MS3	Resolução	Passos por volta
LOW	LOW	LOW	Passo inteiro (é o que acontece com os três soltos)	200
HIGH	LOW	LOW	1/2 passo	400
LOW	HIGH	LOW	1/4 de passo	800
HIGH	HIGH	LOW	1/8 de passo	1.600
HIGH	HIGH	HIGH	1/16 de passo	3.200

No teste e na simulação foi usado o **passo inteiro**. Na máquina de verdade vale usar 1/16, por um motivo explicado na seção 5.2.

> **CUIDADO**
>
> - **Nunca conecte nem desconecte o motor com o driver ligado.** Segundo a Pololu, fabricante do módulo original, isso pode destruir o driver. Desligue tudo antes de mexer nos fios do motor.
> - **Confira a posição do módulo antes de encaixar.** Invertido, ele queima no instante em que a fonte é ligada.
> - **Na protoboard, o módulo tem que atravessar o vão do meio.** Se as duas fileiras de pinos ficarem do mesmo lado, as trilhas internas da protoboard ligam os pinos entre si.

2.5 Ajustando a corrente do driver

O A4988 tem um parafusinho, o **trimpot**, que define o **limite de corrente** que vai para as bobinas:

- **Alto demais:** motor e driver esquentam muito e podem queimar.
- **Baixo demais:** o motor fica fraco, vibra sem girar e perde passos.

Jeito 1: no "olhômetro" (o que vocês fizeram)

1. Gire o trimpot todo para o mínimo (na maioria dos módulos, sentido anti-horário).
2. Ligue o circuito com o código de teste rodando. O motor provavelmente vai ficar fraco ou só vibrar.
3. Aumente o trimpot **aos pouquinhos** (sentido horário), testando, até o motor girar firme.
4. Toque no chip do driver depois de um minuto. Morno é normal. Se estiver quente a ponto de não dar para segurar o dedo, diminua.

Jeito 2: com multímetro (preciso)

O trimpot controla uma tensão chamada **Vref** (tensão de referência). O limite de corrente segue a fórmula do fabricante:

$$Vref = 8 \times Rs \times corrente$$

Rs é o valor dos dois resistores pequenininhos perto do chip (marcados "resistores Rs" na Figura 7). O código impresso neles diz o valor: **R100** = 0,100 Ω, **R068** = 0,068 Ω, **R050** = 0,050 Ω. Para os 0,6 A do motor de vocês:

Código no resistor	Rs	Vref para 0,6 A	Onde costuma aparecer
`R100`	0,100 Ω	**0,48 V**	muitos clones
`R068`	0,068 Ω	**0,33 V**	módulos Pololu feitos desde 2017
`R050`	0,050 Ω	**0,24 V**	módulos Pololu mais antigos

1. Desligue tudo e **desconecte o motor** do driver.
2. Ligue o Arduino no USB e ligue a fonte de 12 V.
3. Coloque o multímetro em **tensão contínua** (V com uma linha reta em cima, ⎓).
4. Encoste a ponta **preta** num GND e a ponta **vermelha** no metal do parafusinho do trimpot.
5. Gire o trimpot devagar até o multímetro mostrar o valor da tabela.
6. Desligue tudo e só então conecte o motor de volta.

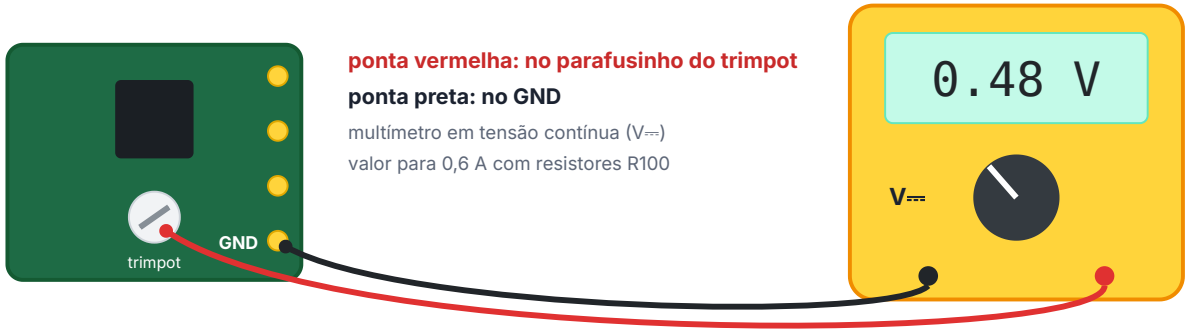


Figura 8 · Medindo a Vref. O multímetro mede a tensão do parafusinho do trimpot. Encoste com cuidado: se a ponta escorregar e encostar em dois pinos ao mesmo tempo, pode dar curto.

Detalhe para os curiosos: no modo passo inteiro, a corrente real em cada bobina fica em cerca de 70% do limite ajustado (informação da Pololu). Então ajustar o limite para a corrente do motor, 0,6 A, deixa uma boa margem de segurança.

2.6 Shield CNC v3: a placa de encaixe

O Shield CNC é uma placa que encaixa **por cima** do Arduino Uno, como um chapéu. Ela tem quatro soquetes para drivers (X, Y, Z e A), um borne para a fonte de 12 V e conectores para chaves de fim de curso. Com ela, não precisa de protoboard nem de fios entre o Arduino e os drivers: as trilhas da placa já fazem as ligações.

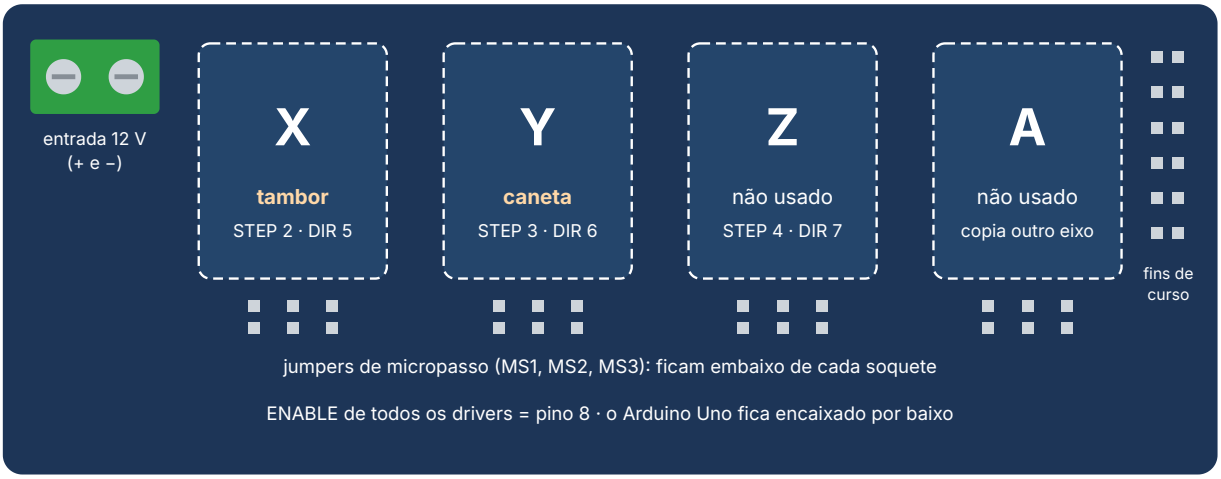


Figura 9 · O Shield CNC v3 (esquema). Cada soquete já está ligado a pinos fixos do Arduino. Por isso a simulação usa exatamente esses números de pino.

Soquete	STEP	DIR	Uso no projeto
X	pino 2	pino 5	motor do **tambor**
Y	pino 3	pino 6	motor da **caneta**
Z	pino 4	pino 7	não usado
A	só copia outro eixo (com jumpers)		não usado
ENABLE de todos os drivers: pino 8 (LOW = ligados)			

Micropasso no Shield: embaixo de cada soquete há três pares de pinos (MS1, MS2 e MS3). Sem nenhum jumper encaixado, o driver fica em passo inteiro. Com os três jumpers, fica em 1/16 de passo.

> **CUIDADO**
>
> Encaixe cada A4988 com o pino **EN** (ENABLE) do driver do mesmo lado em que está escrito "EN" na placa. Ao contrário, o driver queima quando a fonte é ligada.

> **POR QUE O TESTE FOI FEITO NA PROTOBOARD?**
>
> Para testar **um motor sozinho**, a protoboard é mais simples e deixa ver cada fio. A simulação já usa os pinos do Shield, porque é ele que vai na máquina final.

2.7 Servo SG90: o braço da caneta

O servo é um motorzinho com uma caixinha de engrenagens e um circuito de controle dentro. Diferente do motor de passo, ele não gira sem parar: você diz um **ângulo** (de 0° a 180°), ele vai até lá e fica parado, segurando a posição.

Para dizer o ângulo, o Arduino manda um pulso a cada 20 milésimos de segundo, e a **duração do pulso** (entre uns 0,5 e 2,4 milésimos de segundo) indica o ângulo. Você não precisa fazer essa conta: a biblioteca `Servo` faz tudo. Basta escrever `servo.write(90)`.

Fio do SG90	Função	Onde liga
Marrom	GND	GND
Vermelho	+5 V	5V
Laranja	Sinal (o ângulo)	Pino 11 (na simulação)



Figura 10 · O servo em dois ângulos. Os valores 90° e 30° são os da simulação. Na máquina real, vocês ajustam testando, conforme o suporte da caneta.

> **NA PRÁTICA**
>
> Se o Arduino **reiniciar sozinho** quando o servo se mexe, o servo está puxando corrente demais do USB. A solução é alimentar o servo com uma fonte de 5 V separada, mantendo o GND dela ligado ao GND do Arduino.

2.8 Fonte de 12 V e capacitor

A fonte

A fonte dá energia aos motores, pelo pino **VMOT** dos drivers (ou pelo borne do Shield CNC). 12 V é uma tensão comum para NEMA17 com A4988, que aceita de 8 a 35 V. Para dois motores, uma fonte de **12 V e pelo menos 2 A** dá folga.

> **POR QUE NÃO A BATERIA DE 9 V DA LISTA?**
>
> A bateria de 9 V aguenta pouca corrente. Com o motor puxando, a tensão dela cai (fica abaixo dos 8 V que o A4988 precisa), ela descarrega rápido e o motor fica fraco ou perde passos. Serve no máximo para um teste rápido.

O capacitor

O capacitor é como uma **pequena caixa d'água**: guarda um pouco de energia e amortece "picos" de tensão. Quando a corrente nas bobinas do motor muda de repente, elas devolvem um pico de tensão. Sem o capacitor, esse pico pode queimar o driver.

A Pololu recomenda um capacitor eletrolítico de **pelo menos 47 µF** entre VMOT e GND, perto do driver. Vocês usaram **100 µF**, que está ótimo.

CUIDADO: O CAPACITOR TEM LADO CERTO

- A **perna mais comprida** é o **+** e vai no VMOT. A **faixa clara** com sinal de "−" no corpo marca a perna do GND. Invertido, ele pode estufar ou estourar.
- Use um capacitor com tensão **maior** que a da fonte: para 12 V, escolha 25 V ou mais (vem escrito no corpo).
- Alguns Shields CNC já vêm com um capacitor soldado perto do borne de energia. Confira a placa de vocês.

2.9 Tela OLED (só na simulação)

Na simulação, a fita é representada por uma telinha **OLED** de **128 × 64 pontos** (pixels). Cada ponto aceso é um lugar por onde a caneta passou encostada. A tela conversa com o Arduino por **I2C**, um tipo de comunicação que usa só 2 fios: **SDA** (dados, no pino A4) e **SCL** (relógio, no pino A5).

A tela **não faz parte da máquina real**. Ela existe só para a gente enxergar, no computador, o que a caneta desenharia na fita.

EM RESUMO: CADA PEÇA EM UMA LINHA

- **Arduino Uno:** o cérebro; roda o programa e manda ordens (HIGH/LOW) pelos pinos.
- **Motor de passo NEMA17:** gira em passos de 1,8°; 200 passos = 1 volta. Contando pulsos, sabemos onde ele está.
- **Driver A4988:** recebe STEP e DIR do Arduino e entrega a energia da fonte às bobinas. O trimpot limita a corrente.
- **Shield CNC v3:** placa que liga os drivers ao Arduino sem fios. Tambor no X (2/5), caneta no Y (3/6), ENABLE no 8.
- **Servo SG90:** vai para o ângulo pedido; sobe e desce a caneta.
- **Fonte 12 V + capacitor 100 µF:** energia dos motores e proteção dos drivers. GND sempre em comum com o Arduino.

PARTE 3

Montagem e primeiro teste

Esta parte registra o que vocês já fizeram: a ligação de um motor na protoboard, o código que fez ele girar, como gravar pelo VS Code e todos os erros que apareceram no caminho, com a solução de cada um. Serve de consulta sempre que algo der errado.

3.1 A ligação na protoboard

A **protoboard** é uma placa cheia de furinhos que já são ligados por dentro, em pequenas fileiras. Ela serve para montar circuitos sem precisar soldar: é só espetar os fios e os componentes. O módulo A4988 fica **atravessando o vão do meio**, para que cada pino caia numa fileira separada.

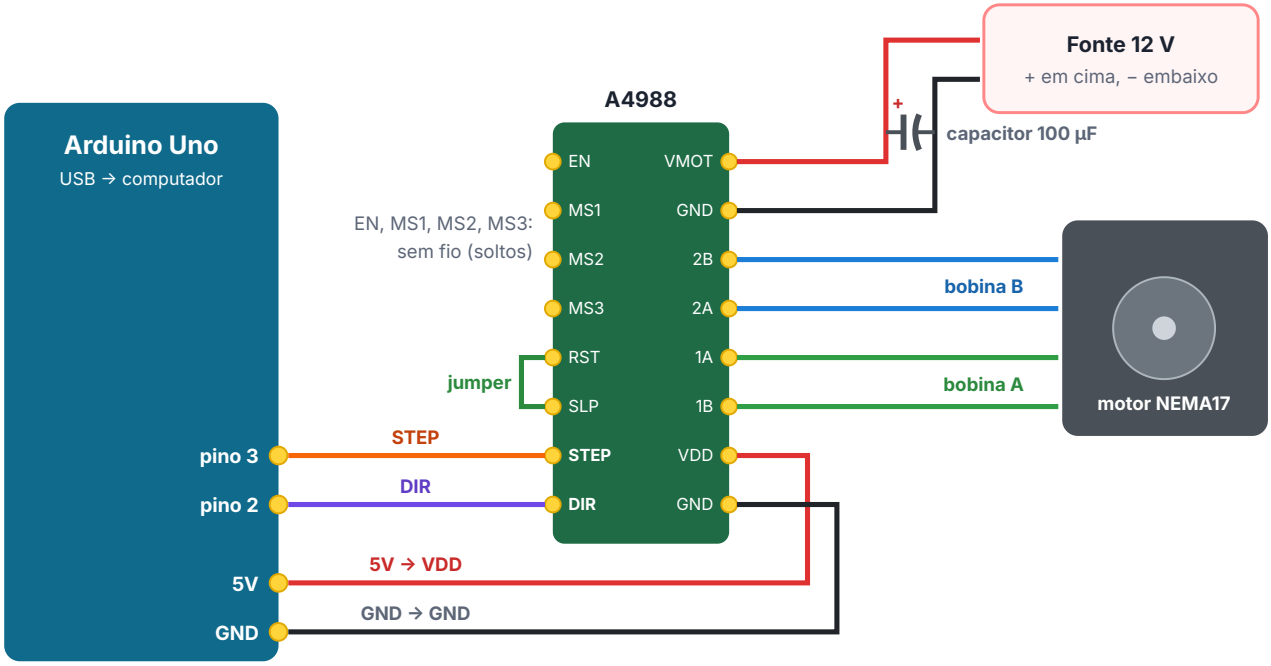


Figura 11 · Todas as ligações do teste com um motor. Desenho em forma de esquema: na protoboard os fios ficam em outras posições, mas as ligações (quem vai em quem) são estas. O "+" do capacitor vai no VMOT.

Use a tabela abaixo como **lista de conferência**: antes de ligar a fonte, confira linha por linha.

#	De	Para	Observação
1	Arduino, pino 3	STEP	os pulsos
2	Arduino, pino 2	DIR	o sentido
3	Arduino, 5V	VDD	lógica do driver
4	Arduino, GND	GND (o de baixo, ao lado do VDD)	
5	RESET	SLEEP	um fio curto ligando os dois
6	EN, MS1, MS2, MS3	nada	soltos: driver ligado, passo inteiro

#	De	Para	Observação
7	Motor, par da bobina A	1A e 1B	par descoberto com multímetro
8	Motor, par da bobina B	2A e 2B	
9	Fonte 12 V, +	VMOT	confira a polaridade duas vezes
10	Fonte 12 V, –	GND (o de cima, ao lado do VMOT)	
11	Capacitor 100 µF	entre VMOT (+) e GND (–)	bem perto do driver

E O GND COMUM?

Os dois pinos GND do módulo A4988 são ligados por dentro da plaquinha. Então, ligando o GND do Arduino num deles e o – da fonte no outro, os dois "chãos" já ficam em comum. Para conferir, use o multímetro no modo continuidade (o que apita): encostando nos dois pinos GND do módulo, ele deve apitar.

3.2 A ordem segura de ligar e desligar

1. Tudo desligado Monte e confira as ligações com a fonte fora da tomada. → **2. USB primeiro** Plugue o Arduino no computador e grave o código. → **3. Monitor Serial** Confira se as mensagens do código aparecem. → **4. Fonte por último** Ligue os 12 V. O motor começa a girar. → **5. Para desligar** Tire a fonte primeiro, e só depois o USB.

POR QUÊ?

Se tiver algum fio errado, a primeira ligação acontece sem os 12 V, que são o que costuma queimar peças. E o motor não dá trancos enquanto o Arduino reinicia durante a gravação.

- **O Arduino precisa continuar ligado no USB** (ou numa fonte de 5 V) enquanto o motor gira. A fonte de 12 V alimenta só os motores, não o Arduino.
- O programa fica gravado. Da próxima vez que o Arduino ligar, ele roda o mesmo programa sozinho, sem precisar do computador.

CUIDADO

Nunca encaixe nem tire os fios do motor com a fonte ligada. É o jeito mais rápido de queimar um A4988.

3.3 O código de teste, linha por linha

Este é o código que fez o motor girar (o arquivo `main.cpp` do PlatformIO). Ele não sabe nada de etiquetas: só gira o motor uma volta para um lado, espera, gira uma volta para o outro, e repete.

MAIN.CPP: COMEÇO

```
1  #include <Arduino.h>
2
3  /*
4    Teste do motor de passo - montagem em protoboard (A4988 + Arduino Uno R3)
5
6    Fiacao:
7      - Arduino 5V      -> VDD do A4988
8      - Arduino GND     -> GND (lado logica) do A4988
9      - Arduino pino 2  -> DIR
10     - Arduino pino 3  -> STEP
11     - SLEEP e RESET do A4988 ligados entre si (jumper)
12     - Motor nos pinos 1A/1B/2A/2B do driver
13     - Fonte 12V no VMOT/GND, com capacitor de 100uF entre eles
14     - GND da fonte tambem ligado ao GND do Arduino
15 */
16
17 void girar(int direcao, int passos);   // avisa o compilador que essa funcao existe
18
19 const int DIR_PIN  = 2;
20 const int STEP_PIN = 3;
21
22 const int PASSOS_POR_TESTE = 200;   // 200 passos = 1 volta (1.8 graus por passo)
23 const int VELOCIDADE_US    = 1000;  // tempo entre pulsos, em microsegundos
```

Linha 1: `#include <Arduino.h>`

Traz para o código as funções do Arduino, como `pinMode` e `digitalWrite`. No Arduino IDE essa linha é colocada sozinha, escondida. No PlatformIO você precisa escrever (foi o primeiro erro que apareceu).

Linhas 3 a 15: comentários

Tudo entre `/*` e `*/`, e tudo depois de `//` até o fim da linha, é **comentário**: texto para pessoas lerem. O computador ignora. Comentários bons explicam o "porquê" do código.

Linha 17: a declaração antecipada

Avisa ao compilador que existe uma função chamada `girar`, escrita mais abaixo no arquivo. O compilador lê de cima para baixo; sem esse aviso, quando ele chega na linha 35 e vê `girar(...)`, reclama que não conhece essa função (foi o segundo erro).

Linhas 19 a 23: as constantes

`const int` cria um **número inteiro que não muda** durante o programa, com um nome. Escrever `DIR_PIN` em vez de `2` deixa o código mais fácil de entender e de mudar: se o fio mudar de pino, você só altera um lugar.

- `PASSOS_POR_TESTE = 200`: 200 passos de 1,8° são uma volta completa.
- `VELOCIDADE_US = 1000`: tempo de espera em **microssegundos** (µs). Um microssegundo é um milionésimo de segundo; 1000 µs = 1 milissegundo.

MAIN.CPP: SETUP() E LOOP()

```
25 void setup() {
26   pinMode(DIR_PIN, OUTPUT);
27   pinMode(STEP_PIN, OUTPUT);
28
29   Serial.begin(9600);
30   Serial.println("Teste do motor de passo iniciado.");
31 }
32
33 void loop() {
34   Serial.println("Sentido horario");
35   girar(HIGH, PASSOS_POR_TESTE);
36   delay(1000);
37
38   Serial.println("Sentido anti-horario");
39   girar(LOW, PASSOS_POR_TESTE);
40   delay(1000);
41 }
```

O `setup()` (linhas 25 a 31)

- `pinMode(DIR_PIN, OUTPUT)`: avisa que esse pino vai **mandar** sinal (saída), e não ler.
- `Serial.begin(9600)`: abre a "conversa" com o computador pelo USB, a 9600 bits por segundo (a **velocidade**, ou *baud rate*). O Monitor Serial tem que estar na mesma velocidade, senão aparecem caracteres estranhos.
- `Serial.println("...")`: escreve uma linha de texto no Monitor Serial. É assim que o Arduino "fala" com a gente.

O `loop()` (linhas 33 a 41)

Avisa no Monitor Serial, chama `girar(HIGH, 200)` (uma volta num sentido), espera `delay(1000)` (1000 milissegundos = 1 segundo), e faz o mesmo no outro sentido com `LOW`. Depois o `loop()` recomeça, para sempre.

MAIN.CPP: A FUNÇÃO GIRAR()

```
43 // Gira o motor um numero de passos numa direcao (HIGH ou LOW)
44 void girar(int direcao, int passos) {
45   digitalWrite(DIR_PIN, direcao);
46
47   for (int i = 0; i < passos; i++) {
48     digitalWrite(STEP_PIN, HIGH);
49     delayMicroseconds(VELOCIDADE_US);
50     digitalWrite(STEP_PIN, LOW);
51     delayMicroseconds(VELOCIDADE_US);
52   }
53 }
```

A função `girar()` (linhas 44 a 53)

Uma **função** é um pedaço de código com nome, que pode ser usado várias vezes. Esta recebe duas informações (os **parâmetros**): `direcao` (HIGH ou LOW) e `passos` (quantos passos dar).

1. **Linha 45:** coloca o pino DIR em HIGH ou LOW, escolhendo o sentido.
2. **Linha 47:** o `for` repete o bloco entre chaves `{ }` várias vezes. A variável `i` começa em 0 e aumenta de 1 em 1 (`i++`) enquanto for menor que `passos`. Com `passos = 200`, o bloco roda 200

vezes (i = 0, 1, 2, ..., 199).

3. **Linhas 48 a 51:** o **pulso**: liga o STEP (HIGH), espera 1000 µs, desliga (LOW), espera 1000 µs. Cada vez que o STEP sobe de LOW para HIGH, o driver dá **1 passo**.

> **ANALOGIA**
>
> O `for` é como um professor de educação física dizendo "faça 200 polichinelos" e contando: zero, um, dois... até 199. Cada polichinelo é um pulso no STEP.

Fazendo as contas: cada passo leva 1000 + 1000 = 2000 µs = 2 ms. Uma volta são 200 passos: 200 × 2 ms = 400 ms = **0,4 segundo por volta**. Ou seja, 150 voltas por minuto.

> **EXPERIMENTE**
>
> - Troque `VELOCIDADE_US` para `3000`: o motor gira 3 vezes mais devagar (e com mais força). É a primeira coisa a tentar quando ele só vibra.
> - Troque para `500`: gira mais rápido, mas pode começar a perder passos.
> - Troque `PASSOS_POR_TESTE` para `50`: o motor gira só 1/4 de volta para cada lado.

3.4 Usando o PlatformIO no VS Code

O **VS Code** é um editor de código. Sozinho, ele não sabe nada de Arduino. A extensão **PlatformIO** ensina o VS Code a compilar (traduzir o código para a linguagem do chip) e gravar no Arduino.

1. No VS Code, abra a aba de extensões (ícone de quadradinhos), procure **PlatformIO IDE** e instale. Deixe reiniciar quando pedir.
2. Clique no ícone da formiguinha na barra lateral. Em **PIO Home**, clique em **New Project**.
3. Preencha: **Name** (ex.: `teste_motor`), **Board**: *Arduino Uno*, **Framework**: *Arduino*. Clique em Finish.
4. Abra `src/main.cpp`, apague tudo e cole o código.
5. Ajuste o `platformio.ini` (abaixo) e use a **barra azul de baixo** para compilar e gravar.

Pasta ou arquivo	O que é
`src/main.cpp`	O código. É aqui que vocês trabalham.
`platformio.ini`	As configurações: placa, porta, velocidade.
`include/, lib/, test/`	Pastas para projetos maiores. Por enquanto, podem ignorar.
`.pio/`	Arquivos que o PlatformIO gera ao compilar. Não mexa.

PLATFORMIO.INI QUE FUNCIONOU

```
[env:uno]
platform = atmelavr
board = uno
framework = arduino
; porta onde o Arduino aparece no Windows
upload_port = COM3
; para o Monitor Serial: mesma porta e mesma velocidade do Serial.begin(9600)
monitor_port = COM3
monitor_speed = 9600
; só se aparecer o erro "not in sync" (alguns clones): apague o ; da linha abaixo
; upload_speed = 57600
```

Na **barra azul** de baixo: ✔ só compila; → compila e grava; a **lixeira** limpa os arquivos gerados; a **tomada** abre o Monitor Serial. Quando a gravação dá certo, o terminal mostra **SUCCESS** em verde.

Diferença	Arduino IDE	PlatformIO
Arquivo do código	`.ino`	`src/main.cpp`
`#include <Arduino.h>`	colocado sozinho	você escreve
Usar uma função antes de escrevê-la	funciona (o IDE cria a declaração)	precisa da declaração antecipada
Escolher a porta	menu Ferramentas → Porta	`upload_port` no `platformio.ini`

Descobrindo a porta: com o Arduino plugado, abra o Gerenciador de Dispositivos do Windows e veja **Portas (COM e LPT)**. Ou rode no terminal do PlatformIO o comando `pio device list`. No terminal comum do Windows (PowerShell), use o caminho completo:

POWERSHELL

```
& "C:\Users\<seu usuário>\.platformio\penv\Scripts\platformio.exe" device list

COM3
----
Hardware ID: USB VID:PID=1A86:7523 SER= LOCATION=1-4
Description: USB-SERIAL CH340 (COM3)
```

3.5 Problemas que já apareceram (e como resolver)

Todos estes aconteceram de verdade durante o projeto. Quando algo der errado, procure a mensagem aqui primeiro.

Ao gravar o código (VS Code e PlatformIO)

O que aparece	O que significa	Como resolver
Não existe botão de gravar; aviso "Configure IntelliSense"; vários erros sublinhados	O arquivo foi aberto solto, sem um projeto PlatformIO	Criar o projeto pelo PIO Home (New Project) e colar o código em `src/main.cpp`
`'OUTPUT' was not declared in this scope` (e também pinMode, Serial, HIGH...)	Faltou trazer as funções do Arduino	Escrever `#include <Arduino.h>` na primeira linha
`'girar' was not declared in this scope`	A função foi usada antes de aparecer no arquivo	Declaração antecipada no topo: `void girar(int direcao, int passos);`

O que aparece	O que significa	Como resolver
`Please specify upload_port`	O PlatformIO não achou a placa sozinho	Descobrir a porta e colocar `upload_port = COM3` no `platformio.ini`
`pio : O termo 'pio' não é reconhecido`	O terminal comum não conhece o comando	Usar o terminal do PlatformIO (ícone na barra azul) ou o caminho completo do `platformio.exe`
`stk500_getsync(): not in sync` ou `programmer is not responding`	O computador achou a porta, mas a placa não respondeu	Na ordem: fechar Arduino IDE e Monitor Serial abertos; nada ligado nos pinos 0 e 1; tirar e pôr o USB; apertar RESET quando aparecer "Uploading"; colocar `upload_speed = 57600`; trocar o cabo USB
Monitor Serial mostra caracteres estranhos	Velocidades diferentes	`monitor_speed = 9600`, igual ao `Serial.begin(9600)`

Com o motor ligado

O que acontece	Causas prováveis e soluções (teste nesta ordem)
Vibra mas não gira	1) velocidade alta demais: `VELOCIDADE_US = 3000`; 2) pares de fios do motor trocados: refaça o teste dos pares; 3) corrente baixa: suba o trimpot aos poucos; 4) fonte fraca (bateria de 9 V) ou sem GND comum; 5) mau contato na protoboard: aperte os fios
Não faz nada, nem vibra	Fonte desligada ou invertida; falta o jumper entre RESET e SLEEP; ENABLE ligado em HIGH; fios de STEP e DIR em pinos diferentes dos do código; Arduino sem o código gravado
Gira para o lado contrário	Normal. Inverta os dois fios de **uma** das bobinas (troque 1A com 1B), ou troque HIGH por LOW no código
Driver muito quente	Corrente alta demais: diminua o trimpot (seção 2.5). Um dissipador pequeno também ajuda
Arduino reinicia sozinho	Algo puxando corrente demais do USB (geralmente o servo): use uma fonte de 5 V separada para ele, com GND comum

No Wokwi

O que aparece	Como resolver
Painel da simulação todo preto, sem peças e sem botão de play	O `diagram.json` ficou com texto a mais (por exemplo, sobrou a primeira linha do conteúdo antigo). Clique na aba, `Ctrl+A`, apague e cole de novo
Erro de biblioteca não encontrada	Confira se existe o arquivo `libraries.txt`, com esse nome exato
As letras aparecem pequenas demais	Dê zoom com a roda do mouse em cima do painel da simulação

3.6 Os códigos que vocês têm

Arquivo	Para quê	Pinos	Situação
`main.cpp` (= `teste_motor_protoboard.ino`)	Testar 1 motor na protoboard: uma volta para cada lado	DIR 2, STEP 3	testado

Arquivo	Para quê	Pinos	Situação
`teste_motores_passo.ino`	Testar os 2 motores no Shield CNC, um de cada vez	tambor STEP 2 / DIR 5; caneta STEP 3 / DIR 6; ENABLE 8	espera o Shield

`sketch.ino`
`diagram.json`
`libraries.txt`
	Simulação no Wokwi da impressora completa (Parte 4)	os do Shield + servo no 11 + tela em A4/A5	funcionando

CUIDADO: O PINO 2 MUDA DE PAPEL

No teste da protoboard, o pino 2 é o **DIR**. No Shield CNC (e na simulação), o pino 2 é o **STEP** do tambor, e o DIR fica no 5. Quando passarem para o Shield, usem os códigos feitos para ele.

PARTE 4

A simulação no Wokwi

Enquanto as peças não chegam, a simulação permite desenvolver e testar a parte mais importante do projeto: a lógica que transforma uma palavra em movimentos dos motores e da caneta. Esta parte explica o que a simulação mostra e, depois, o código inteiro, bloco por bloco.

4.1 O que é o Wokwi

O **Wokwi** (wokwi.com) é um simulador de eletrônica que roda no navegador, de graça. Você monta o circuito com peças virtuais, cola o código, aperta o play, e o Wokwi roda o código **como se fosse um Arduino de verdade**.

Para o projeto, isso tem duas vantagens: dá para trabalhar sem as peças, e não tem como queimar nada. E o mesmo código depois vai rodar no Arduino real, só tirando a parte da tela.

A simulação testa	A simulação não testa
Se a lógica do código está certa	Corrente, aquecimento e o ajuste do trimpot
Se os pinos estão ligados nos lugares certos	A fonte de 12 V (o Wokwi ignora o VMOT)
A ordem dos movimentos: motores, servo, caneta	Força dos motores e perda de passos
O desenho que sairia na fita	A mecânica: correia, tambor, suporte da caneta
As mensagens no Monitor Serial	O Bluetooth

4.2 O que cada peça da simulação representa

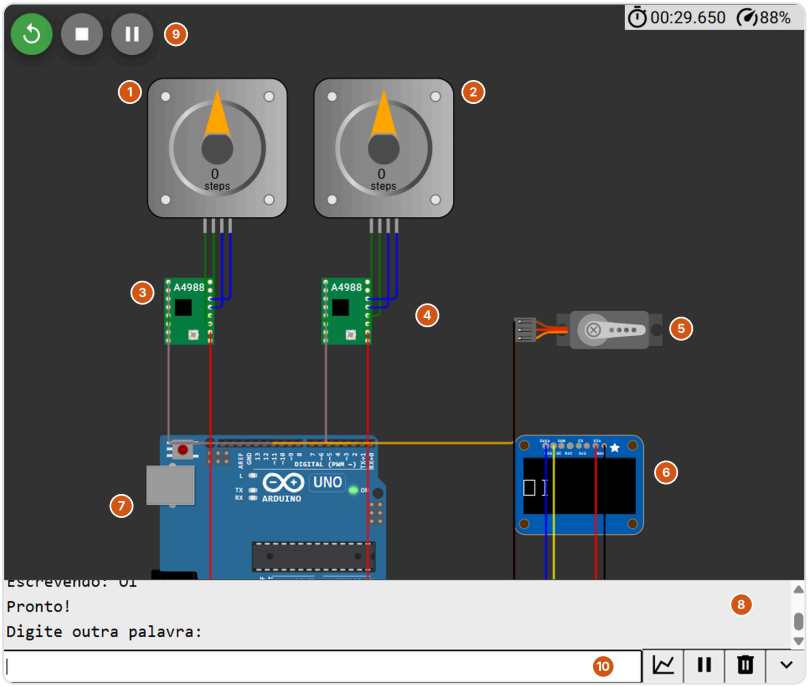


Figura 12 · A simulação de vocês rodando (print do dia 9 de outubro). Na tela azul, à direita, aparece a palavra "OI" que tinha sido digitada. As letras são pequenas; dê zoom no Wokwi para ver melhor.

Nº	Na simulação	Na máquina real
1	Motor da esquerda (`motorTambor`)	O motor que gira o rolo de fita
2	Motor da direita (`motorCaneta`)	O motor que move a caneta pela correia
3	Driver da esquerda (`drvTambor`)	O A4988 no soquete X do Shield
4	Driver da direita (`drvCaneta`)	O A4988 no soquete Y do Shield
5	Servo (`servoCaneta`)	O SG90 que encosta e afasta a caneta
6	Tela azul (`fita`)	A fita da etiqueta: o que a caneta desenharia
7	Arduino Uno	O mesmo Arduino de vocês
8	Serial Monitor (mensagens)	O Monitor Serial do VS Code
9	Botões reiniciar, parar e pausar	(só da simulação)
10	Campo de texto: onde se digita a palavra	No futuro, o aplicativo pelo Bluetooth

O número em cada motor ("0 steps") é a **contagem de passos**: aumenta quando ele gira para um lado e diminui quando gira para o outro. A seta laranja mostra para onde o eixo aponta. No print, os dois mostram 0 porque a máquina já tinha terminado a palavra e voltado para o ponto de partida.

4.3 Como montar e rodar a simulação

1. Entre em **wokwi.com** e faça login (para poder salvar).
2. Crie um projeto novo escolhendo **Arduino Uno**.
3. Na aba `sketch.ino` : `Ctrl+A` , apague e cole o código.
4. Na aba `diagram.json` : `Ctrl+A` , apague **tudo, inclusive a primeira linha**, e cole o conteúdo do arquivo.
5. Crie o arquivo de bibliotecas: na seta ao lado das abas, escolha **New file**, dê o nome `libraries.txt` e cole as três linhas abaixo.
6. Clique em **SAVE** e depois no **play** verde.
7. Quando aparecer "Digite uma palavra e aperte Enter:", clique no campo de texto embaixo do Serial Monitor, digite e aperte Enter.

LIBRARIES.TXT

```
Adafruit SSD1306
Adafruit GFX Library
Servo
```

4.4 O que você deve ver

1. No Serial Monitor: "Impressora de etiquetas pronta." e "Escrevendo: UEFS".
2. Os números dos motores mudando, as setas girando, o bracinho do servo subindo e descendo, e as letras aparecendo na tela, **ponto por ponto**.
3. No final: "Pronto!", os motores voltam para o 0, e aparece "Digite uma palavra e aperte Enter:".

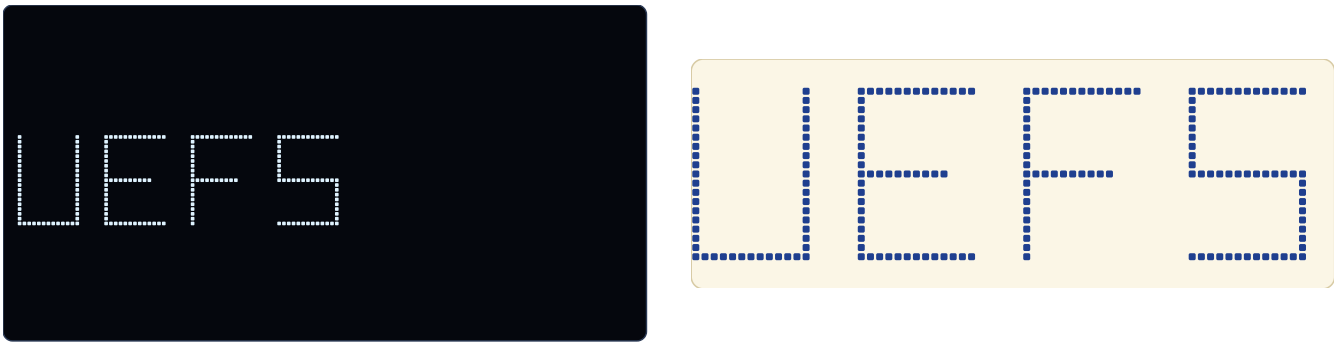


Figura 13 · O resultado de "UEFS", gerado rodando o código. À esquerda, a tela inteira (128 × 64 pontos), como no Wokwi. À direita, o mesmo desenho ampliado, como ficaria a tinta na fita.

AS REGRAS DO TEXTO

- No máximo **7 letras** por etiqueta (o motivo está na seção 4.6).
- Letras aceitas: **A B C D E F G H I L M N O P R S T U V** e o espaço. Minúsculas viram maiúsculas sozinhas.
- Uma letra fora da lista é pulada: aparece "Letra nao suportada: X" no Serial Monitor e fica um espaço vazio no lugar dela.
- A simulação pode rodar um pouco mais devagar que o tempo real. O Wokwi mostra a porcentagem no canto (no print, 88%).

4.5 Visão geral do código

O código tem 293 linhas. Parece muito, mas ele é organizado em blocos pequenos, e cada bloco faz uma coisa só:

Linhas	Bloco	O que faz
1–19	Comentário inicial	Explica o projeto (o computador ignora)
21–24	Bibliotecas	Trazem código pronto: tela e servo
26–42	Constantes	Os pinos e os ajustes
44–52	Objetos e variáveis	A tela, o servo e a "memória" da máquina
54–74	`setup()`	Prepara tudo e escreve "UEFS"
76–89	`loop()`	Espera uma palavra no Serial Monitor
93–111	`escreverTexto()`	Palavra → letras
115–195	`desenharLetra()`	Letra → retas
198–213	`linha()`	Reta → posições dos motores
218–240	`irPara()`	Anda em linha reta, ponto por ponto
242–268	`passoTambor()`, `passoCaneta()`, `pulso()`	Um passo de cada vez
272–293	`abaixarCaneta()`, `levantarCaneta()`, `marcarFita()`	O servo e a tela

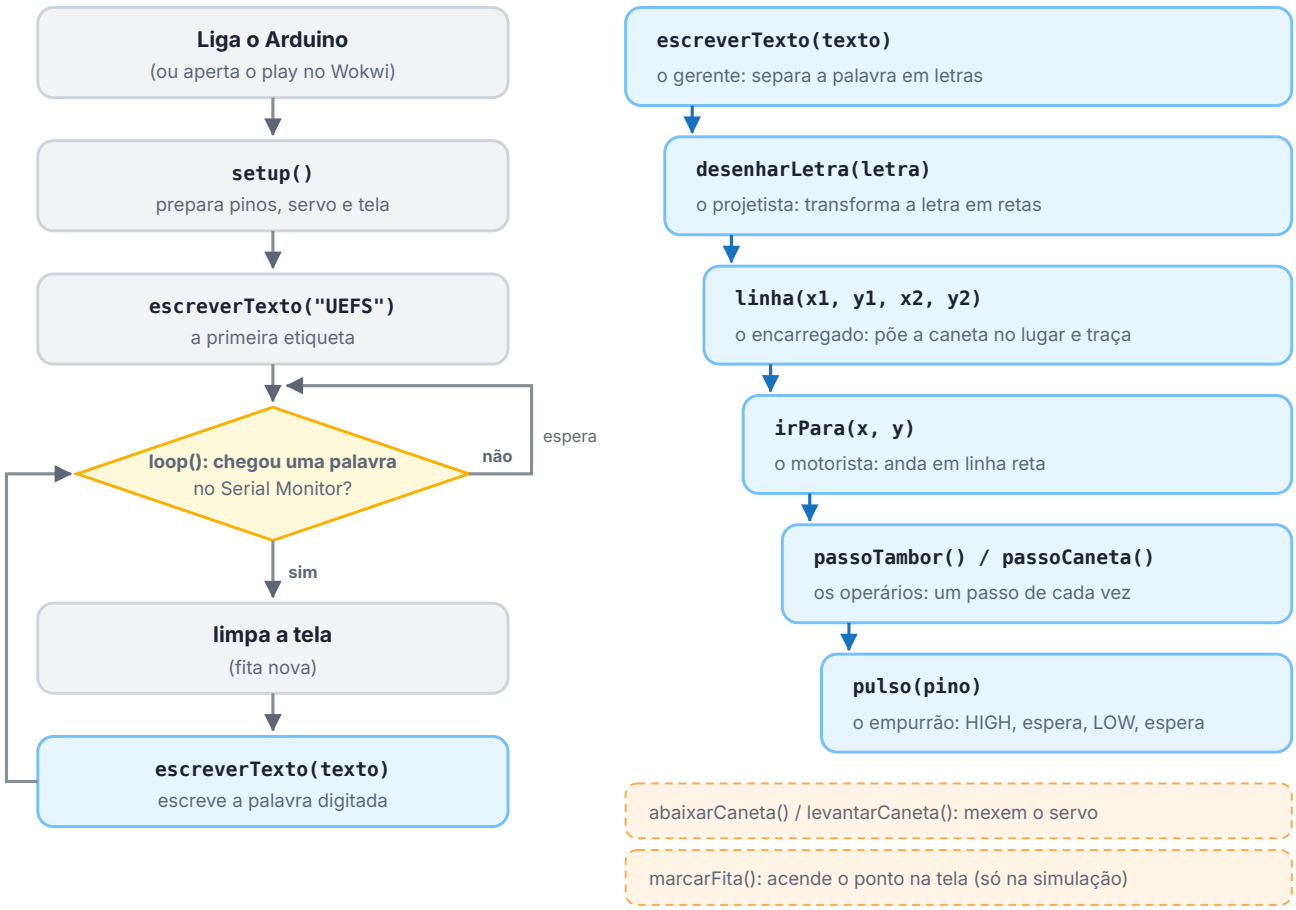


Figura 14 · À esquerda, a ordem em que as coisas acontecem; depois de cada palavra, o programa volta a esperar. **À direita, quem chama quem**: cada função entrega um pedaço menor do trabalho para a de baixo. As funções do servo são usadas por `linha()` e `escreverTexto()`.

ANALOGIA: UMA PEQUENA EMPRESA

O **gerente** (`escreverTexto`) recebe o pedido "UEFS" e distribui uma letra de cada vez. O **projetista** (`desenharLetra`) sabe de cor as retas de cada letra. O **encarregado** (`linha`) põe a caneta no começo de cada reta. O **motorista** (`irPara`) leva a caneta em linha reta até o fim. Os **operários** (`passoTambor` , `passoCaneta` , `pulso`) dão cada passinho. Ninguém precisa saber o trabalho dos outros: só o seu.

4.6 Bibliotecas, pinos e ajustes

SKETCH.INO: BIBLIOTECAS, CONSTANTES E OBJETOS

```
21 #include <Wire.h>
22 #include <Adafruit_GFX.h>
23 #include <Adafruit_SSD1306.h>
24 #include <Servo.h>
25
26 // ---------- Pinos (mesmos do Shield CNC v3) ----------
27 const int TAMBOR_STEP = 2;   // eixo X do shield
28 const int TAMBOR_DIR  = 5;
29 const int CANETA_STEP = 3;   // eixo Y do shield
30 const int CANETA_DIR  = 6;
31 const int ENABLE_PIN  = 8;   // LOW = drivers ligados
32 const int SERVO_PIN   = 11;
33
34 // ---------- Ajustes ----------
35 const int CANETA_CIMA  = 90;   // angulo do servo com a caneta levantada
36 const int CANETA_BAIXO = 30;   // angulo do servo com a caneta encostando na fita
37
38 const int ESCALA = 3;          // quantos passos (pixels) vale 1 quadradinho da letra
39 const int ESPACO_LETRA = 6;    // cada letra ocupa 6 quadradinhos (4 de letra + 2 de espaco)
40 const int MAX_LETRAS = 7;      // cabe na tela de 128 pixels
41 const int BASE = 42;           // linha de baixo das letras na tela
42 const int TEMPO_PASSO_US = 1500;
43
44 // ---------- Objetos ----------
45 Adafruit_SSD1306 tela(128, 64, &Wire, -1);
46 Servo servo;
```

Bibliotecas (linhas 21 a 24)

Uma **biblioteca** é um pacote de código pronto, escrito por outras pessoas. Com `#include`, você usa as funções dela sem precisar saber como funcionam por dentro.

- `Wire.h`: a comunicação I2C (os 2 fios da tela).
- `Adafruit_GFX.h` e `Adafruit_SSD1306.h`: desenhar na tela OLED.
- `Servo.h`: controlar o servo com `servo.write(ângulo)`.

Os pinos (linhas 27 a 32)

São os pinos do Shield CNC (seção 2.6): tambor no soquete X (STEP 2, DIR 5), caneta no soquete Y (STEP 3, DIR 6), ENABLE no 8. O servo fica no 11.

Os ajustes (linhas 35 a 42)

Constante	Valor	O que significa	Se você mudar...
`CANETA_CIMA`	90	Ângulo do servo com a caneta levantada	Ajustar na máquina real
`CANETA_BAIXO`	30	Ângulo com a caneta encostada	Ajustar na máquina real
`ESCALA`	3	Cada quadradinho da grade da letra vale 3 passos (3 pontos)	Maior = letras maiores

Constante	Valor	O que significa	Se você mudar...
`ESPACO_LETRA`	6	Cada letra ocupa 6 quadradinhos: 4 de letra + 2 de espaço	Maior = letras mais separadas
`MAX_LETRAS`	7	Quantas letras cabem na tela	Tem que caber nos 128 pontos
`BASE`	42	Linha da tela onde fica o "pé" das letras	Sobe ou desce o texto
`TEMPO_PASSO_US`	1500	Metade do tempo de cada passo, em µs	Maior = mais devagar

> **POR QUE NO MÁXIMO 7 LETRAS?**
>
> Cada letra ocupa ESPACO_LETRA × ESCALA = 6 × 3 = **18 pontos** de largura. Sete letras ocupam 7 × 18 = 126 pontos, e a tela tem 128. Uma oitava letra começaria no ponto 126 e não caberia.

Os objetos (linhas 45 e 46)

`Adafruit_SSD1306 tela(128, 64, &Wire, -1);` cria a tela: 128 por 64 pontos, conversando pelo I2C (`Wire`); o `-1` quer dizer que ela não usa pino de reset. `Servo servo;` cria o servo. Daqui para frente, o código usa os nomes `tela` e `servo` para mandar neles.

4.7 A memória da máquina: as variáveis de posição

SKETCH.INO: POSIÇÃO ATUAL DA MÁQUINA

```
48 // ---------- Posicao atual da maquina ----------
49 int posTambor = 0;        // em passos (= pixel horizontal na tela)
50 int posCaneta = BASE;     // em passos (= pixel vertical na tela)
51 bool canetaAbaixada = false;
52 int inicioLetra = 0;      // onde comeca a letra que esta sendo escrita
```

Uma **variável** é uma caixinha com nome que guarda um valor que **pode mudar** (ao contrário das constantes). `int` guarda números inteiros; `bool` guarda verdadeiro (`true`) ou falso (`false`).

Lembra que o Arduino não tem nenhum sensor de posição? Ele sabe onde a caneta está porque **conta cada passo que mandou**: cada passo para um lado soma 1, cada passo para o outro subtrai 1. É como andar de olhos fechados contando os passos: se você contou certo, sabe onde está.

- `posTambor`: quantos passos o tambor está do ponto de partida. Na simulação, é o **x** (horizontal) na tela.
- `posCaneta`: a posição da caneta. Na simulação, é o **y** (vertical). Começa em `BASE` (42), o pé das letras.
- `canetaAbaixada`: `true` se a caneta está encostada na fita. Começa `false` (levantada).
- `inicioLetra`: em que x começa a letra que está sendo desenhada.

Na simulação, **1 passo = 1 ponto da tela**. A figura abaixo mostra o "mapa" da tela:

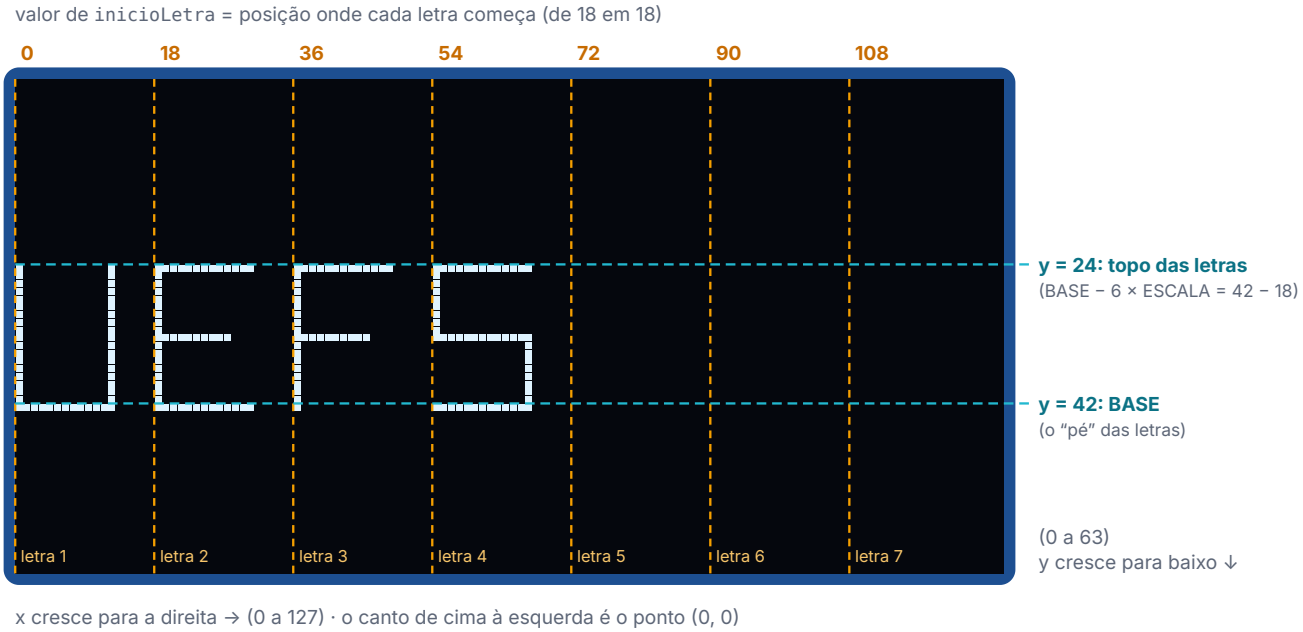


Figura 15 · O mapa da tela. Cada letra começa 18 pontos depois da anterior. O pé das letras fica na linha 42 e o topo na linha 24.

ATENÇÃO: NA TELA, Y CRESCE PARA BAIXO

O ponto (0, 0) é o canto de **cima** à esquerda, e o y aumenta descendo. Então, para a caneta **subir** na tela, o y **diminui**. Parece estranho, mas é o padrão de quase todas as telas de computador.

4.8 setup() e loop()

SKETCH.INO: SETUP()

```
54 void setup() {
55   pinMode(TAMBOR_STEP, OUTPUT);
56   pinMode(TAMBOR_DIR, OUTPUT);
57   pinMode(CANETA_STEP, OUTPUT);
58   pinMode(CANETA_DIR, OUTPUT);
59   pinMode(ENABLE_PIN, OUTPUT);
60   digitalWrite(ENABLE_PIN, LOW);   // liga os dois drivers
61
62   servo.attach(SERVO_PIN);
63   servo.write(CANETA_CIMA);
64
65   Serial.begin(9600);
66
67   tela.begin(SSD1306_SWITCHCAPVCC, 0x3C);
68   tela.clearDisplay();
69   tela.display();
70
71   Serial.println("Impressora de etiquetas pronta.");
72   escreverTexto("UEFS");
73   Serial.println("Digite uma palavra e aperte Enter:");
74 }
```

- **Linhas 55 a 59:** os cinco pinos dos drivers viram saídas.
- **Linha 60:** ENABLE em LOW: **liga os dois drivers** (no A4988, LOW = ligado).

- **Linhas 62 e 63:** liga o servo no pino 11 e manda para 90°: caneta levantada.
- **Linha 65:** abre o Serial Monitor a 9600.
- **Linha 67:** liga a tela. O `0x3C` é o **endereço** dela no I2C: vários aparelhos podem dividir os mesmos 2 fios, e cada um tem um endereço, como o número de uma casa na rua.
- **Linhas 68 e 69:** apaga a tela. Detalhe importante: a tela só mostra as mudanças quando o código chama `tela.display()`.
- **Linha 72:** escreve a primeira etiqueta, "UEFS".

SKETCH.INO: LOOP()

```
76 void loop() {
77   if (Serial.available() > 0) {
78     String texto = Serial.readStringUntil('\n');
79     texto.trim();
80     texto.toUpperCase();
81
82     if (texto.length() > 0) {
83       tela.clearDisplay();   // "fita nova"
84       tela.display();
85       escreverTexto(texto);
86       Serial.println("Digite outra palavra:");
87     }
88   }
89 }
```

- **Linha 77:** `Serial.available() > 0` pergunta: "tem alguma coisa digitada esperando para ser lida?". Se não tem, o `loop()` termina e começa de novo, milhares de vezes por segundo, até alguém digitar.
- **Linha 78:** lê o que foi digitado até o Enter (`'\n'` é o caractere do Enter) e guarda numa variável do tipo `String` (texto).
- **Linhas 79 e 80:** `trim()` tira espaços e o Enter das pontas; `toUpperCase()` passa tudo para maiúsculas ("uefs" vira "UEFS").
- **Linhas 82 a 86:** se sobrou alguma coisa, limpa a tela ("fita nova") e escreve a palavra.

4.9 escreverTexto(): da palavra para as letras

SKETCH.INO: ESCREVERTEXTO()

```
 93 void escreverTexto(String texto) {
 94   if (texto.length() > MAX_LETRAS) {
 95     Serial.println("Texto grande demais, vou escrever so as 7 primeiras letras.");
 96   }
 97
 98   Serial.print("Escrevendo: ");
 99   Serial.println(texto);
100
101   int quantidade = texto.length();
102   for (int i = 0; i < quantidade && i < MAX_LETRAS; i++) {
103     inicioLetra = i * ESPACO_LETRA * ESCALA;
104     desenharLetra(texto.charAt(i));
105   }
106
107   // terminou: levanta a caneta e volta pro comeco
108   levantarCaneta();
109   irPara(0, BASE);
110   Serial.println("Pronto!");
111 }
```

O `for` da linha 102 passa por cada letra. O `&&` significa "e": ele continua enquanto `i` for menor que a quantidade de letras **e** menor que 7. Para cada letra, calcula onde ela começa (linha 103) e pede para `desenharLetra()` desenhá-la.

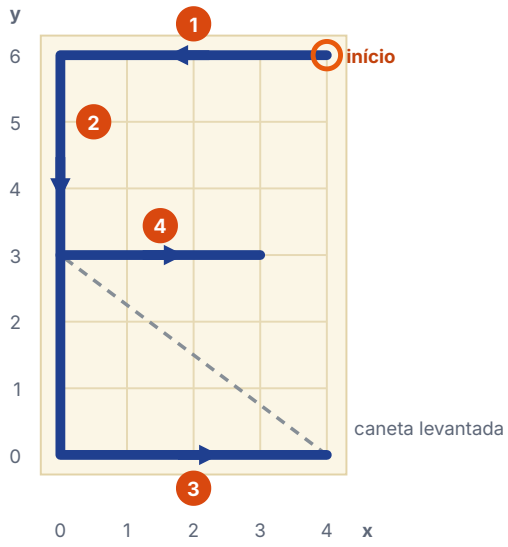
Letra	1ª	2ª	3ª	4ª	5ª	6ª	7ª
valor de `i`	0	1	2	3	4	5	6
`inicioLetra` = i × 18	0	18	36	54	72	90	108

> **A CONTAGEM COMEÇA DO ZERO**
>
> Em programação, a primeira posição de um texto é a 0, não a 1. Em "UEFS", `charAt(0)` é o "U" e `charAt(3)` é o "S".

No final (linhas 108 a 110), a caneta sobe e volta para o ponto de partida (0, BASE). É por isso que os motores do Wokwi voltam a mostrar "0 steps" quando a palavra termina.

4.10 desenharLetra(): da letra para as retas

Cada letra é desenhada numa **grade de 4 de largura por 6 de altura**. O ponto (0, 0) é o canto de baixo à esquerda e (4, 6) é o canto de cima à direita. Cada letra é uma lista de retas, e cada reta é escrita como `linha(x1, y1, x2, y2)`: vai do ponto (x1, y1) até o ponto (x2, y2) da grade.



O código da letra E

```
case 'E':
  linha(4, 6, 0, 6);   // 1
  linha(0, 6, 0, 0);   // 2
  linha(0, 0, 4, 0);   // 3
  linha(0, 3, 3, 3);   // 4
  break;
```

As retas 1, 2 e 3 são emendadas: a caneta não levanta.
Antes da reta 4, a caneta sobe e vai até (0, 3) sem riscar.
As setinhas mostram o sentido em que cada reta é traçada.

Figura 16 · A letra E na grade 4 × 6. Os números mostram a ordem das retas. A linha tracejada é a caneta se movendo levantada.

O `switch` compara a letra recebida com cada `case`. Quando encontra a letra certa, executa as linhas dela até o `break`. É como um cardápio: pediu "E", a cozinha faz a receita do E. Veja o fim da função:

SKETCH.INO: FINAL DE DESENHARLETRA()

```
185     case 'V':
186       linha(0, 6, 2, 0); linha(2, 0, 4, 6);
187       break;
188     case ' ':
189       break;   // espaco: nao desenha nada
190     default:
191       Serial.print("Letra nao suportada: ");
192       Serial.println(letra);
193       break;
194   }
195 }
```

O `case ' '` (espaço) não desenha nada: só deixa o lugar vazio. O `default` pega qualquer letra que não tem receita: avisa no Serial Monitor e não desenha nada.

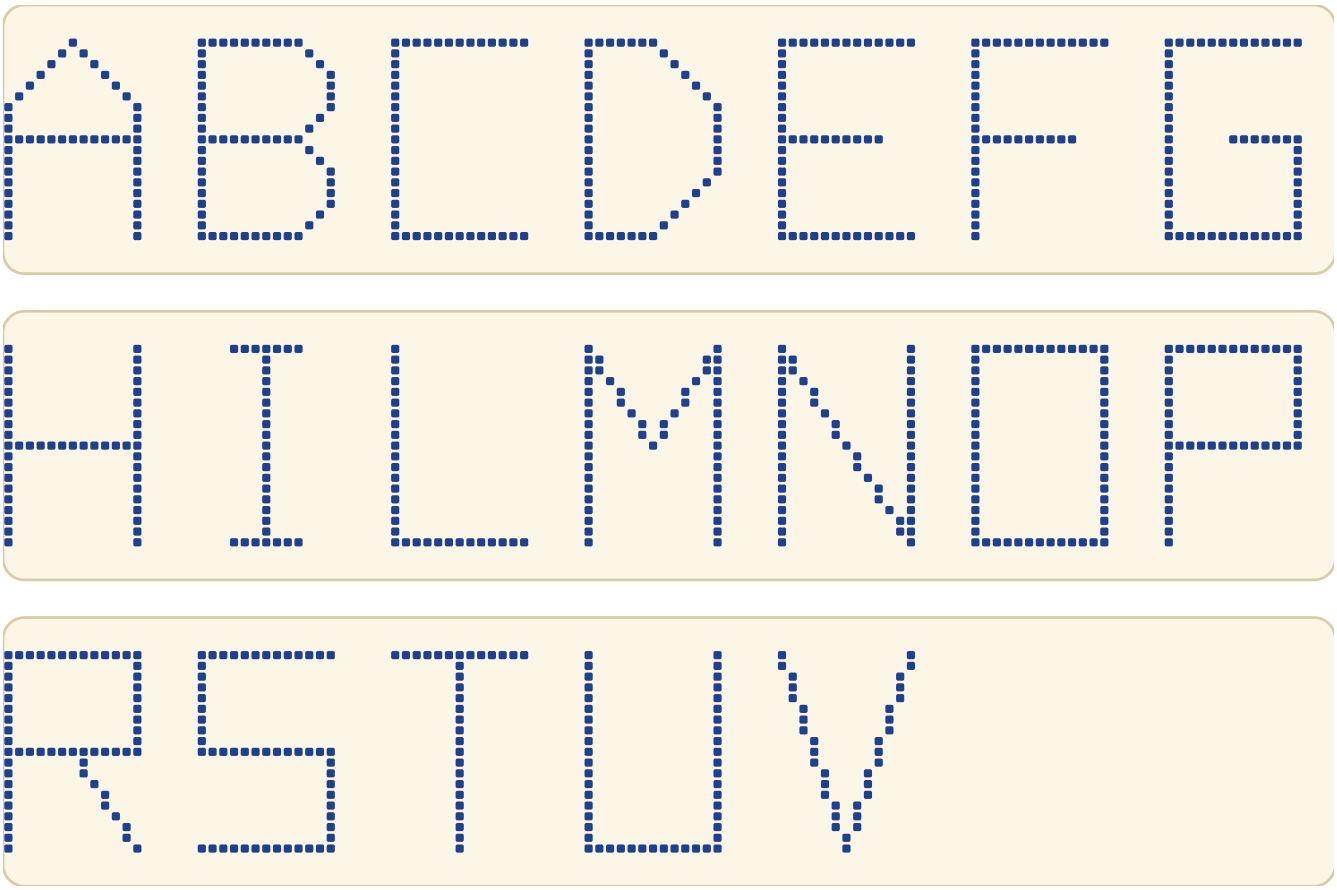


Figura 17 · Todas as 19 letras que o código sabe desenhar, geradas rodando o próprio código (três etiquetas: "ABCDEFG", "HILMNOP" e "RSTUV").

4.11 linha(): da grade para os passos

SKETCH.INO: LINHA()

```
197 // Desenha uma reta entre dois pontos da grade da letra
198 void linha(int x1, int y1, int x2, int y2) {
199   // converte da grade da letra para passos dos motores
200   int inicioX = inicioLetra + x1 * ESCALA;
201   int inicioY = BASE - y1 * ESCALA;
202   int fimX = inicioLetra + x2 * ESCALA;
203   int fimY = BASE - y2 * ESCALA;
204
205   // se a caneta nao esta no comeco da linha, levanta e vai ate la
206   if (posTambor != inicioX || posCaneta != inicioY || !canetaAbaixada) {
207     levantarCaneta();
208     irPara(inicioX, inicioY);
209     abaixarCaneta();
210   }
211
212   irPara(fimX, fimY);
213 }
```

As linhas 200 a 203 convertem um ponto da grade da letra para a posição dos motores (os pontos da tela):

$$x = \text{inicioLetra} + x_{grade} \times \text{ESCALA} \quad \text{e} \quad y = \text{BASE} - y_{grade} \times \text{ESCALA}$$

Por que o y tem um sinal de **menos**? Na grade da letra, o y cresce para **cima** (0 é o pé, 6 é o topo). Na tela, o y cresce para **baixo**. O "menos" inverte o sentido. Exemplo com o E sendo a **2ª letra** (`inicioLetra` = 18, ESCALA = 3, BASE = 42):

Ponto da grade	Conta do x	x	Conta do y	y
(4, 6): canto de cima à direita	18 + 4 × 3	**30**	42 − 6 × 3	**24**
(0, 6): canto de cima à esquerda	18 + 0 × 3	**18**	42 − 6 × 3	**24**
(0, 0): canto de baixo à esquerda	18 + 0 × 3	**18**	42 − 0 × 3	**42**
(4, 0): canto de baixo à direita	18 + 4 × 3	**30**	42 − 0 × 3	**42**
(0, 3): meio, à esquerda	18 + 0 × 3	**18**	42 − 3 × 3	**33**
(3, 3): fim do traço do meio	18 + 3 × 3	**27**	42 − 3 × 3	**33**

A linha 206 decide se a caneta precisa ser reposicionada. Os símbolos: `!=` é "diferente de", `||` é "ou", e `!` é "não". Em português: *se o tambor não está no começo da reta, **ou** a caneta não está no começo da reta, **ou** a caneta não está abaixada*, então levanta a caneta, vai até o começo sem riscar e abaixa. Se a reta nova começa exatamente onde a anterior terminou, com a caneta já abaixada, ela segue direto, sem levantar. Isso deixa o desenho mais rápido e os cantos mais bonitos.

4.12 irPara(): andando em linha reta

SKETCH.INO: IRPARA()

```
217 // Move os dois motores juntos ate o ponto (xAlvo, yAlvo) em linha reta
218 void irPara(int xAlvo, int yAlvo) {
219   int xInicio = posTambor;
220   int yInicio = posCaneta;
221   int dx = xAlvo - xInicio;
222   int dy = yAlvo - yInicio;
223
224   // quantos pontinhos tem no caminho (o maior dos dois deslocamentos)
225   int pontos = max(abs(dx), abs(dy));
226
227   for (int i = 1; i <= pontos; i++) {
228     int x = xInicio + round((float)dx * i / pontos);
229     int y = yInicio + round((float)dy * i / pontos);
230
231     while (posTambor < x) passoTambor(1);
232     while (posTambor > x) passoTambor(-1);
233     while (posCaneta < y) passoCaneta(1);
234     while (posCaneta > y) passoCaneta(-1);
235
236     if (canetaAbaixada) {
237       marcarFita();
238     }
239   }
240 }
```

Esta é a função mais esperta do código. Para ir de um ponto a outro em linha reta, ela divide o caminho em **pontinhos** e, a cada pontinho, move o motor que precisar mover:

1. **Linhas 221 e 222:** calcula quanto falta andar em cada direção: `dx` (tambor) e `dy` (caneta).

2. **Linha 225:** o número de pontinhos é o **maior** dos dois deslocamentos (`abs` tira o sinal de menos: abs(−18) = 18).
3. **Linhas 228 e 229:** calcula onde cada motor deveria estar no pontinho `i`, em proporção.
4. **Linhas 231 a 234:** os `while` ("enquanto") dão passos até cada motor chegar lá.
5. **Linhas 236 a 238:** se a caneta está abaixada, marca o ponto na tela.

Exemplo: a primeira reta do V, de (0, 24) até (6, 42). Aí `dx` = 6 e `dy` = 18, então são 18 pontinhos. A caneta anda em todos; o tambor anda só de vez em quando:

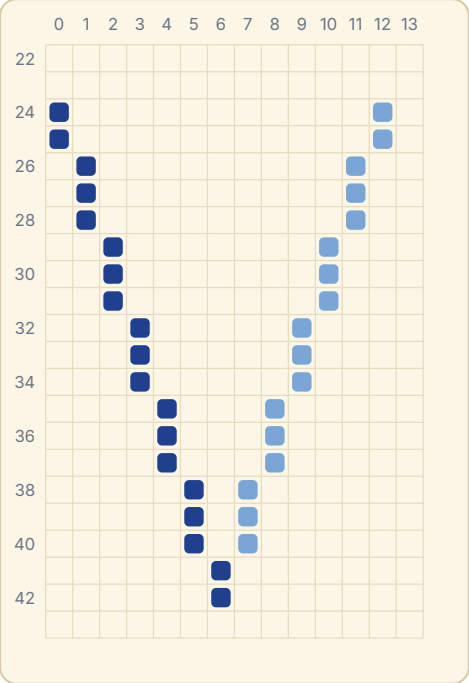


Figura 18 · O V ampliado. Azul-escuro: a primeira reta. Azul-claro: a segunda. Repare na "escadinha".

i	Conta do x	x	y	Quem andou
1	6 × 1 ÷ 18 = 0,33 → 0	0	25	caneta
2	6 × 2 ÷ 18 = 0,67 → 1	1	26	tambor e caneta
3	6 × 3 ÷ 18 = 1,00 → 1	1	27	caneta
4	6 × 4 ÷ 18 = 1,33 → 1	1	28	caneta
5	6 × 5 ÷ 18 = 1,67 → 2	2	29	tambor e caneta
6	6 × 6 ÷ 18 = 2,00 → 2	2	30	caneta
…	…	…	…	…
18	6 × 18 ÷ 18 = 6,00 → 6	6	42	chegou!

`round()` arredonda para o inteiro mais perto: 0,33 vira 0; 0,67 vira 1.

O `(float)` na linha 228 transforma a conta em número com vírgula. Sem ele, o Arduino faria uma divisão de inteiros, que joga fora a parte depois da vírgula: 6 × 2 ÷ 18 daria 0 em vez de 0,67.

ANALOGIA

É como atravessar na diagonal um chão de lajotas pisando só no centro delas: às vezes você avança uma lajota para a frente, às vezes uma para a frente e uma para o lado. De longe, o caminho parece uma reta; de perto, é uma escadinha. As máquinas CNC de verdade fazem algo bem parecido (o GRBL, por exemplo, usa um método chamado algoritmo de Bresenham).

4.13 Os passos, o pulso e a caneta

SKETCH.INO: PASSOTAMBOR(), PASSOCANETA() E PULSO()

```
242 void passoTambor(int direcao) {
243   if (direcao > 0) {
244     digitalWrite(TAMBOR_DIR, HIGH);
245   } else {
246     digitalWrite(TAMBOR_DIR, LOW);
247   }
248   pulso(TAMBOR_STEP);
249   posTambor = posTambor + direcao;
250 }
251
252 void passoCaneta(int direcao) {
253   if (direcao > 0) {
254     digitalWrite(CANETA_DIR, HIGH);
255   } else {
256     digitalWrite(CANETA_DIR, LOW);
257   }
258   pulso(CANETA_STEP);
259   posCaneta = posCaneta + direcao;
260 }
261
262 // Um pulso no STEP = o motor anda 1 passo
263 void pulso(int pinoStep) {
264   digitalWrite(pinoStep, HIGH);
265   delayMicroseconds(TEMPO_PASSO_US);
266   digitalWrite(pinoStep, LOW);
267   delayMicroseconds(TEMPO_PASSO_US);
268 }
```

`passoTambor(1)` dá um passo para um lado: põe o DIR em HIGH, dá o pulso e soma 1 em `posTambor`. `passoTambor(-1)` faz o contrário: DIR em LOW e subtrai 1. A `passoCaneta()` é igual, para o outro motor. A `pulso()` é o "empurrão" no driver:

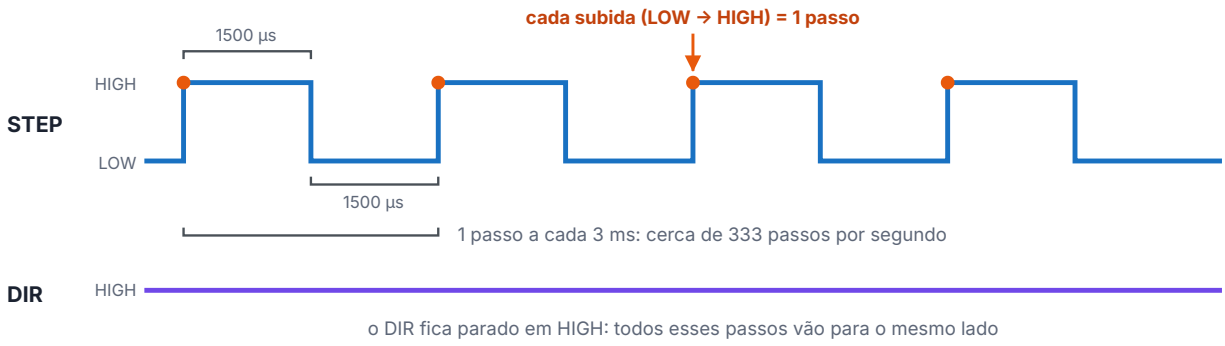


Figura 19 · Os sinais que o Arduino manda para o driver enquanto o motor anda 4 passos para o mesmo lado. O driver dá um passo a cada subida do STEP.

Com `TEMPO_PASSO_US` = 1500, cada passo leva 1500 + 1500 = 3000 µs = 3 ms. Uma volta inteira do motor (200 passos) levaria 0,6 segundo.

SKETCH.INO: A CANETA E A TELA

```
// ================== CANETA ==================

void abaixarCaneta() {
  if (!canetaAbaixada) {
    servo.write(CANETA_BAIXO);
    delay(200);
    canetaAbaixada = true;
    marcarFita();
  }
}

void levantarCaneta() {
  if (canetaAbaixada) {
    servo.write(CANETA_CIMA);
    delay(200);
    canetaAbaixada = false;
  }
}

// So existe na simulacao: acende o ponto onde a caneta esta
void marcarFita() {
  tela.drawPixel(posTambor, posCaneta, SSD1306_WHITE);
  tela.display();
}
```

- O `if (!canetaAbaixada)` evita mandar o servo para onde ele já está.
- O `delay(200)` espera 0,2 segundo para o servo chegar ao ângulo **antes** de os motores andarem. Sem essa espera, a caneta encostaria atrasada (falhando o começo da reta) ou levantaria atrasada (riscando o caminho).
- A `marcarFita()` acende o ponto onde a caneta está e manda a tela se atualizar. Como ela é chamada a cada pontinho, a letra aparece aos poucos, como se estivesse sendo escrita.

4.14 Acompanhando a letra L do começo ao fim

Juntando tudo: o que acontece, em ordem, quando você digita **L** e aperta Enter. A letra L tem só duas retas: `linha(0, 6, 0, 0)` e `linha(0, 0, 4, 0)`. Por ser a 1ª letra, `inicioLetra` = 0.

#	O que o código faz	Servo	Motor da caneta	Motor do tambor	Tela
1	`linha(0, 6, 0, 0)` calcula: começo (0, 24), fim (0, 42)	90°	—	—	—
2	A caneta está levantada e longe do começo: `levantarCaneta()` não faz nada, ela já está em cima	90°	—	—	—
3	`irPara(0, 24)`: sobe até o topo do L	90°	18 passos, DIR LOW	—	nada (caneta levantada)
4	`abaixarCaneta()`: servo a 30°, espera 0,2 s	30°	—	—	acende o ponto (0, 24)
5	`irPara(0, 42)`: desce traçando	30°	18 passos, DIR HIGH	—	acende 18 pontos: o traço vertical

#	O que o código faz	Servo	Motor da caneta	Motor do tambor	Tela
6	`linha(0, 0, 4, 0)` : o começo (0, 42) é onde a caneta já está, abaixada. Não levanta!	30°	—	—	—
7	`irPara(12, 42)` : vai para a direita traçando	30°	—	12 passos, DIR HIGH	acende 12 pontos: o traço horizontal
8	Acabou o texto: `levantarCaneta()`	90°	—	—	—
9	`irPara(0, 42)` : volta ao ponto de partida	90°	—	12 passos, DIR LOW	nada

Total: 36 passos do motor da caneta, 24 do tambor e 31 pontos acesos (1 + 18 + 12). Esses números foram conferidos rodando o código: batem exatamente.

4.15 Exercício: criando letras novas

Faltam letras como J, K, Q, W, X, Y, Z e os números. Criar uma é um ótimo exercício para o grupo, e não precisa mexer em mais nada do código:

1. Desenhe a letra numa grade 4 × 6 no papel (tem grades em branco abaixo).
2. Liste as retas na ordem. Tente **emendar**: o fim de uma reta ser o começo da próxima. Assim a caneta não precisa levantar.
3. Escreva o `case` dentro do `switch`, antes do `case ' ':`.
4. Teste no Wokwi digitando a letra nova.

Exemplo resolvido, o J e o K:

DUAS LETRAS NOVAS (COLE ANTES DO CASE ' ':)

```
case 'J':
  linha(1, 6, 4, 6); linha(4, 6, 4, 0); linha(4, 0, 0, 0); linha(0, 0, 0, 2);
  break;
case 'K':
  linha(0, 0, 0, 6);
  linha(4, 6, 0, 3); linha(0, 3, 4, 0);
  break;
```

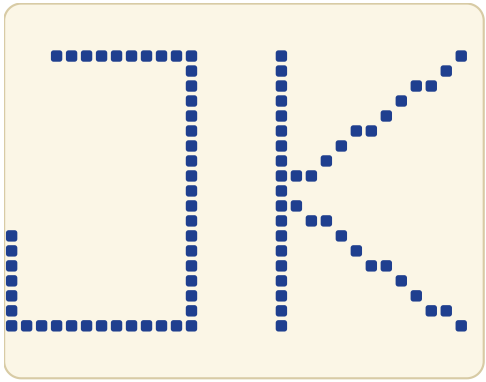


Figura 20 · O resultado do J e do K, gerado rodando o código com essas linhas a mais. O J é feito de uma tirada só; o K precisa levantar a caneta uma vez.

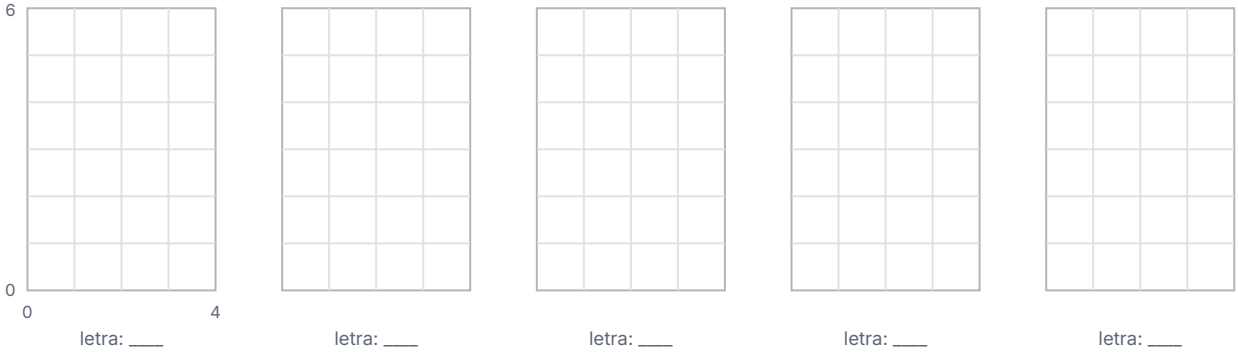


Figura 21 · Grades em branco para planejar letras. Os cantos são (0, 0) embaixo à esquerda e (4, 6) em cima à direita.

DESAFIOS PARA O GRUPO

- **Fácil:** criar as letras que faltam (Q, W, X, Y, Z) e os números de 0 a 9.
- **Médio:** mudar a `ESCALA` para 4. As letras ficam com 24 pontos de altura e 24 de largura por letra: quantas cabem na tela? (Resposta: 128 ÷ 24 = 5,3, então `MAX_LETRAS` precisa virar 5.)
- **Difícil:** desenhar uma moldura em volta da etiqueta, usando `linha()` ou `irPara()`.

PARTE 5

Da simulação para a máquina de verdade

A simulação resolveu a lógica. Agora falta o mundo físico: medidas em milímetros, o firmware GRBL, o servo de verdade, o Bluetooth e a mecânica. Esta parte explica o que muda e propõe um roteiro de metas para o grupo.

5.1 O que continua igual e o que muda

Continua igual	Muda
A lógica: `escreverTexto`, `desenharLetra`, `linha` e `irPara`	**Sai a tela:** bibliotecas, objeto `tela` e o conteúdo de `marcarFita()`
Os pinos do Shield CNC	**Ângulos do servo:** ajustar testando no suporte real
A ideia de contar passos para saber a posição	**Velocidade:** depende dos motores reais, com peso
O jeito de andar em linha reta	**Tamanho:** passos viram milímetros, e cada eixo tem o seu valor (seção 5.2)
	Energia: fonte de 12 V, capacitor e ajuste do trimpot entram em cena
	Ponto de partida: na simulação a caneta "nasce" em (0, 42); na máquina, ela precisa de uma posição conhecida ao ligar (ver "fim de curso" no glossário)

Tirando a tela do código

1. Apague as linhas 21, 22 e 23 (as bibliotecas da tela). Mantenha a linha 24 (`Servo.h`).
2. Apague a linha 45 (o objeto `tela`).
3. Apague as linhas 67, 68 e 69 (ligar e limpar a tela, no `setup()`) e as linhas 83 e 84 (limpar a tela, no `loop()`).
4. Deixe a função `marcarFita()` vazia: apague as linhas 291 e 292. Assim não precisa mexer em quem chama ela.

E, se for gravar pelo **PlatformIO**, lembre das duas lições da seção 3.4: `#include <Arduino.h>` na primeira linha e uma declaração antecipada para cada função, logo depois:

PARA O MAIN.CPP DO PLATFORMIO (LOGO DEPOIS DOS #INCLUDE)

```
void escreverTexto(String texto);
void desenharLetra(char letra);
void linha(int x1, int y1, int x2, int y2);
void irPara(int xAlvo, int yAlvo);
void passoTambor(int direcao);
void passoCaneta(int direcao);
void pulso(int pinoStep);
void abaixarCaneta();
void levantarCaneta();
void marcarFita();
```

5.2 Calibração: quantos passos é um milímetro?

Na simulação, 1 passo é 1 ponto da tela, nos dois eixos. Na máquina real, o que importa é **quantos milímetros a caneta anda a cada passo**. E isso é diferente em cada eixo, porque cada motor move a fita ou a caneta de um jeito.

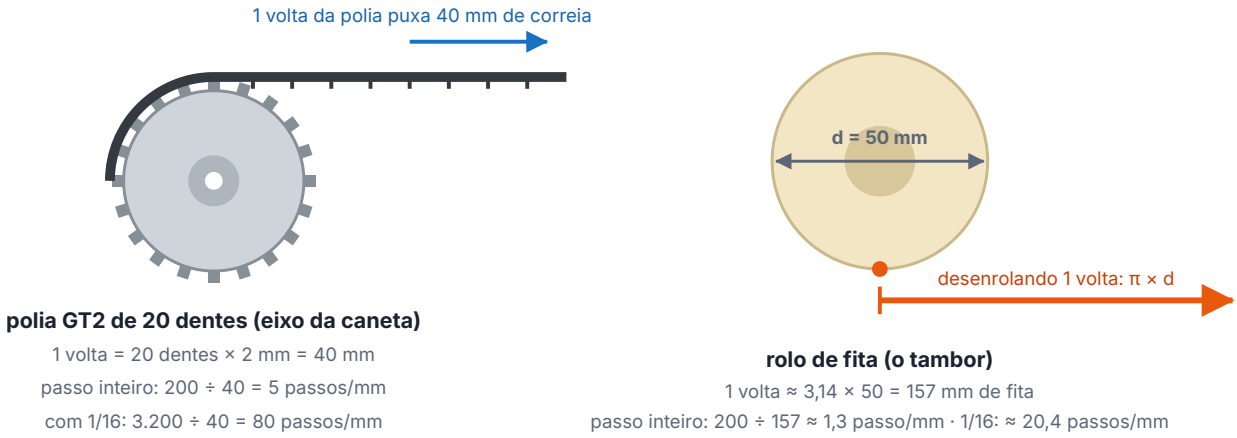


Figura 22 · Os dois eixos andam distâncias diferentes por passo. O diâmetro de 50 mm é só um exemplo: meçam o rolo de vocês.

Eixo	Conta	Passo inteiro	1/16 de passo
Caneta (correia GT2, polia de 20 dentes)	passos por volta ÷ 40 mm	5 passos/mm (0,2 mm por passo)	80 passos/mm (0,0125 mm por passo)
Tambor (rolo de 50 mm, exemplo)	passos por volta ÷ (3,14 × diâmetro)	≈ 1,3 passo/mm (≈ 0,8 mm por passo)	≈ 20,4 passos/mm (≈ 0,05 mm por passo)

POR QUE USAR 1/16 DE PASSO NA MÁQUINA REAL?

Com passo inteiro, cada passo do tambor move a fita quase **1 mm**. As letras ficariam grosseiras, "serrilhadas". Com 1/16, cada passo move só 0,05 mm. Por isso vale encaixar os três jumpers de micropasso embaixo de cada driver no Shield (seção 2.6).

Agora veja o que aconteceria se o código da simulação fosse para a máquina **sem nenhuma mudança**:

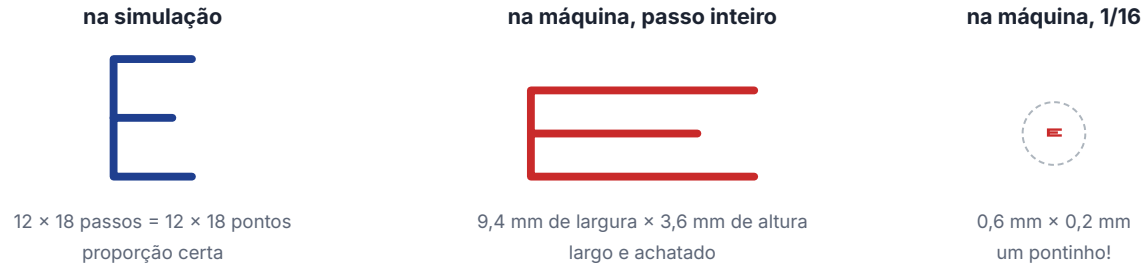


Figura 23 · A mesma letra E (12 × 18 passos) em três situações. Sem converter, a letra sai deformada ou minúscula, porque cada eixo tem um "passos por milímetro" diferente.

Conclusão: na máquina real, o código precisa pensar em **milímetros**. Para uma letra de 4 × 6 mm, ele calcula 4 × (passos/mm do tambor) passos para um eixo e 6 × (passos/mm da caneta) para o outro. O GRBL, explicado a seguir, faz essa conversão sozinho.

NA PRÁTICA: MEDINDO EM VEZ DE CALCULAR

1. Marque a posição da caneta (ou um ponto no rolo).
2. Mande o motor andar um número conhecido de passos, por exemplo 800.
3. Meça com uma régua quantos milímetros andou, por exemplo 10 mm.
4. Divida: 800 ÷ 10 = **80 passos por mm**.

A medida pega detalhes que a conta não pega, como o rolo ser um pouco mais grosso ou fino do que o previsto.

O ROLO VAI AFINANDO

Conforme a fita é usada, o rolo fica mais fino, e cada volta anda menos fita. Para etiquetas pequenas a diferença é pequena, mas vale medir o diâmetro de vez em quando e corrigir o valor, ou aceitar letras um pouco mais estreitas no fim do rolo.

5.3 GRBL e G-code

O **GRBL** é um **firmware** (um programa para o Arduino) gratuito, feito para controlar máquinas CNC. Ele é gravado uma vez no Arduino e, a partir daí, o Arduino passa a obedecer comandos de **G-code** que chegam pelo USB ou pelo Bluetooth. O GRBL já resolve, prontinho:

- retas e curvas, com os dois motores coordenados;
- aceleração suave (o motor não dá tranco nem perde passos ao arrancar);
- a conversão de milímetros para passos em cada eixo;
- fins de curso e posição inicial.

O código da simulação é uma versão simples e didática do que o GRBL faz por dentro: a nossa `irPara()` é parecida com o comando `G1`. Entender a simulação ajuda a entender o GRBL.

A "língua" G-code

G-code é uma lista de comandos, um por linha, usada por quase todas as máquinas CNC e impressoras 3D. O que vem depois de `;` é comentário.

Comando	O que faz
`G21`	As medidas estão em milímetros
`G90`	Coordenadas absolutas: X10 é "o ponto 10", e não "ande mais 10"
`G0 X.. Y..`	Vai rápido até o ponto (usado com a caneta levantada)
`G1 X.. Y.. F..`	Vai em linha reta até o ponto, com velocidade F (em mm por minuto)
`M3 / M5`	No GRBL com servo: abaixa / levanta a caneta (veja abaixo)
`$100=, $101=`	Configuram os passos por mm do eixo X e do eixo Y

A LETRA L EM G-CODE (LETRA DE 4 × 6 MM)

```
G21             ; medidas em milímetros
G90             ; coordenadas absolutas
M5              ; caneta levantada
G0 X0 Y6        ; vai até o topo do L, sem riscar
M3              ; abaixa a caneta
G1 X0 Y0 F600   ; desce traçando, a 600 mm por minuto
G1 X4 Y0        ; traço horizontal
M5              ; levanta a caneta
G0 X0 Y0        ; volta ao começo
```

Compare com a tabela da seção 4.14: é a mesma sequência, só que escrita em G-code e em milímetros. A calibração da seção 5.2 vira duas configurações do GRBL, digitadas uma vez:

EXEMPLO DE CALIBRAÇÃO NO GRBL (COM 1/16 DE PASSO)

```
$100=20.4       ; eixo X = tambor (rolo de 50 mm: medir o de vocês!)
$101=80         ; eixo Y = caneta (correia GT2, polia de 20 dentes)
```

O servo no GRBL

O GRBL normal não controla servo. O pino 11 dele (o mesmo que usamos para o servo na simulação) gera um sinal feito para controlar a velocidade do motor de uma fresadora ou a potência de um laser, rápido demais para um servo entender. Existem **versões modificadas do GRBL** que trocam isso por controle de servo: nelas, `M3` desce e `M5` sobe a caneta. Em algumas, o comando leva um valor, como `M3 S90`. Escolher a versão é tarefa da etapa do servo.

No Shield CNC v3, o pino 11 aparece no conector marcado como fim de curso do eixo Z (Z+/Z−), por causa de uma troca de pinos feita nas versões mais novas do GRBL. Confirmem isso quando chegarem nessa etapa.

Mandando o G-code

- **Pelo computador:** o programa gratuito **Universal G-code Sender** (UGS) manda G-code pelo USB e tem botões para mover os eixos na mão. Ótimo para os primeiros testes.
- **Pelo celular:** o aplicativo **GRBL Controller** (Android), citado no documento do projeto, manda o G-code pelo Bluetooth (seção 5.4).

E quem transforma "UEFS" em G-code?

Alguém precisa traduzir o texto em comandos. Duas opções:

1. **Inkscape** (programa de desenho gratuito) com uma extensão que gera G-code a partir de um texto ou desenho.
2. **Um programa próprio** no computador, em Python ou Java, usando o mesmo "alfabeto de retas" da simulação. Cada `linha(x1, y1, x2, y2)` vira quatro comandos: `G0` até o começo, `M3`, `G1` até o fim e `M5`. Seria uma parte de software do projeto feita inteira pelo grupo.

5.4 Bluetooth e o aplicativo

O módulo **HC-05** cria uma "porta serial sem fio": o que o celular manda chega ao Arduino como se viesse pelo cabo USB. Ele não estava na lista de componentes, então precisa ser comprado.

Pino do HC-05	Liga em	Observação
VCC	5V	
GND	GND	
TXD	pino 0 (RX) do Arduino	o que o HC-05 transmite, o Arduino recebe
RXD	pino 1 (TX) do Arduino, **com divisor de tensão**	a entrada do HC-05 trabalha com 3,3 V: TX do Arduino → resistor de 1 kΩ → RXD; e RXD → resistor de 2 kΩ → GND

CUIDADO: DOIS DETALHES QUE TRAVAM MUITA GENTE

- **Pinos 0 e 1 são os mesmos do USB.** Desconecte o HC-05 sempre que for gravar código, senão aparece o erro "not in sync" (seção 3.5).
- **Velocidade:** o GRBL conversa a 115200; o HC-05 normalmente vem de fábrica a 9600. É preciso configurar o HC-05 para 115200 com um comando AT, como `AT+UART=115200,0,0`. Sem isso, o aplicativo conecta, mas a máquina não entende nada.

No celular: pareie com o HC-05 nas configurações de Bluetooth (a senha costuma ser 1234 ou 0000), abra o GRBL Controller, conecte e mande o G-code.

5.5 Roteiro das próximas metas

#	Meta	Como saber que deu certo
0	feito Testar um motor com o A4988 na protoboard	O motor gira uma volta para cada lado
1	feito Simulação da impressora no Wokwi	A palavra digitada aparece na tela
2	agora Sem peças: criar as letras que faltam (4.15); desenhar no Tinkercad as peças a imprimir (encaixe da caneta no carrinho, encaixe do rolo); fechar a lista de compras (5.6)	Letras novas funcionando no Wokwi; peças desenhadas; lista com preços
3	Com o Shield e o segundo driver: testar o segundo motor; montar no Shield; ajustar a Vref dos dois drivers; rodar `teste_motores_passo.ino`	Os dois motores giram, cada um no seu soquete
4	Gravar o GRBL e controlar pelo Universal G-code Sender; calibrar `$100` e `$101` medindo com régua	`G1 X10` anda 10 mm de verdade
5	Servo da caneta com um GRBL modificado; ajustar os ângulos	`M3` encosta a caneta e `M5` levanta
6	Mecânica: trilho, correia e carrinho; tambor com rolamentos; montar tudo	A caneta anda de ponta a ponta sem travar e o rolo gira sem folga
7	Texto → G-code no computador	A primeira etiqueta de verdade!
8	Bluetooth: HC-05 a 115200 e o aplicativo GRBL Controller	Etiqueta impressa mandando pelo celular

5.6 O que ainda falta comprar

Item	Para quê	Observação
Polia GT2 de 20 dentes, furo de 5 mm	Eixo da caneta	O eixo do NEMA17 tem 5 mm
Correia GT2 (6 mm de largura)	Eixo da caneta	Comprimento: cerca de 2 × o trilho, mais folga
Polia livre (idler) GT2, com rolamento	Esticar a correia na outra ponta	
Suporte para NEMA17	Prender o motor no trilho	Pode ser impresso em 3D
Fonte de 12 V, 2 A ou mais	Energia dos motores	Substitui a bateria de 9 V
Módulo Bluetooth HC-05	Controle pelo celular	Não estava na lista original
Resistores de 1 kΩ e 2 kΩ	Divisor de tensão do HC-05	
Chave de fim de curso (micro switch)	Posição inicial da caneta	Opcional, mas recomendado
Capacitor de 100 µF, 25 V ou mais	Proteger os drivers	Se o Shield não tiver um
Jumpers de micropasso	Usar 1/16 de passo	Costumam vir com o Shield
Dissipadores para o A4988	Resfriar os drivers	Opcional
Parafusos M3	Fixar os NEMA17	A furação da frente do NEMA17 é M3

CONSULTA

Glossário

As palavras técnicas da apostila, em ordem alfabética. O número entre parênteses é a seção onde o assunto é explicado com mais calma.

A4988
Driver de motor de passo: recebe STEP e DIR do Arduino e alimenta as bobinas com a energia da fonte (2.4).

Arduino Uno
Placa com o microcontrolador ATmega328P; o cérebro do projeto (2.2).

Baud rate (velocidade)
Quantos bits por segundo passam na comunicação serial. Os dois lados precisam usar a mesma: 9600 no código da apostila, 115200 no GRBL.

Biblioteca
Pacote de código pronto, usado com `#include` (4.6).

Bobina
Fio enrolado que vira um ímã quando passa corrente. O NEMA17 tem duas, A e B (2.3).

Bootloader
Programinha já gravado no Arduino que recebe o código novo pelo USB. Alguns clones têm uma versão antiga, que grava a 57600 (3.5).

Capacitor
Componente que guarda um pouco de energia e amortece picos de tensão; protege o driver (2.8).

CH340
Chip que faz a ponte entre o USB e o Arduino nos Unos compatíveis (2.2).

CNC
Comando Numérico Computadorizado: máquina cujos movimentos são comandados por computador.

Compilar
Traduzir o código escrito por pessoas para a linguagem do chip.

Constante
Valor com nome que não muda durante o programa (`const`) (3.3).

Coordenada
Par de números (x, y) que diz onde fica um ponto (1.3).

Corrente
Quantidade de eletricidade que passa; medida em amperes, A (2.1).

Declaração antecipada
Linha que avisa ao compilador que uma função existe e está escrita mais abaixo (3.3).

DIR
Pino do driver que escolhe o sentido do giro (2.4).

Driver
Circuito que recebe sinais fracos e entrega energia forte a um motor (2.4).

ENABLE
Pino do driver que liga (LOW) ou desliga (HIGH) o motor (2.4).

Fim de curso
Chavinha que avisa quando a caneta chegou numa ponta do trilho. Ao ligar, a máquina anda até ela para começar sempre do mesmo ponto (isso se chama *homing*).

Firmware
Programa gravado num aparelho para controlá-lo. O GRBL é um firmware (5.3).

Função
Pedaço de código com nome, que pode ser usado várias vezes e receber parâmetros (3.3).

G-code
A "língua" de comandos das máquinas CNC (5.3).

GND
O 0 V, o "chão" de referência. Tem que ser comum entre as peças que conversam (2.1).

GRBL
Firmware gratuito para Arduino que transforma G-code em movimentos dos motores (5.3).

GT2
Tipo de correia dentada, com um dente a cada 2 mm (5.2).

HC-05
Módulo Bluetooth que funciona como uma porta serial sem fio (5.4).

HIGH e LOW
Os dois estados de um pino digital: 5 V e 0 V (2.1).

I2C
Comunicação de 2 fios (SDA e SCL) usada pela tela da simulação (2.9).

loop()
Função do Arduino que se repete para sempre (2.2).

Micropasso
Dividir cada passo do motor em pedaços menores, como 1/16 (2.4).

Monitor Serial
Janela onde aparecem as mensagens do Arduino e onde se digita texto para ele.

Motor de passo
Motor que gira em passos fixos; 200 por volta no NEMA17 de vocês (2.3).

NEMA17
Tamanho padrão de motor de passo: frente de 42 × 42 mm (2.3).

OLED
Tipo de tela. Na simulação, uma de 128 × 64 pontos faz o papel da fita (2.9).

Passo
O menor movimento do motor de passo: 1,8° nos motores de vocês (2.3).

Passos por milímetro
Quantos passos fazem a caneta (ou a fita) andar 1 mm. É diferente em cada eixo (5.2).

Perder passos
Quando o motor não acompanha os pulsos e a posição real fica diferente da contada (2.3).

Pino
Cada ponto de ligação do Arduino ou de um módulo.

PlatformIO
Extensão do VS Code para programar o Arduino (3.4).

Plotter de tambor
Máquina que desenha com caneta sobre um cilindro que gira (1.1).

Porta COM
O "endereço" do Arduino no Windows, como COM3 (2.2).

Protoboard
Placa com furos ligados por dentro, para montar circuitos sem solda (3.1).

Pulso
Sinal que sobe de LOW para HIGH e volta. No STEP, cada pulso é um passo (4.13).

Servo
Motor que vai para um ângulo pedido e fica parado lá (2.7).

setup()
Função do Arduino que roda uma vez só, ao ligar (2.2).

Shield
Placa que encaixa por cima do Arduino. O Shield CNC v3 recebe os drivers (2.6).

STEP
Pino do driver que recebe os pulsos; cada pulso é um passo (2.4).

Tambor
O cilindro que gira. No projeto, é o próprio rolo de fita (1.1).

Tensão
A "força" que empurra a eletricidade; medida em volts, V (2.1).

Trimpot
Parafusinho do driver que ajusta o limite de corrente (2.5).

Upload (gravar)
Mandar o código do computador para o Arduino.

Variável
Caixinha com nome que guarda um valor que pode mudar (4.7).

VDD e VMOT
No driver: VDD é a alimentação da lógica (5 V do Arduino); VMOT é a dos motores (12 V da fonte) (2.4).

Vref
Tensão do trimpot que define o limite de corrente: Vref = 8 × Rs × corrente (2.5).

Wokwi
Simulador de Arduino que roda no navegador (4.1).

CONSULTA

Referências e links úteis

De onde vieram os dados técnicos desta apostila, e onde aprender mais.

Dados técnicos

1. Documentos do projeto: "Impressora de Etiquetas CNC" e "Componentes etiquetadora CNC" (arquivos do grupo).
2. Pololu, A4988 Stepper Motor Driver Carrier: pinagem, fórmula da Vref, tabela de micropasso, recomendação do capacitor e aviso sobre desconectar o motor. https://www.pololu.com/product/1182
3. Catálogo de motores de passo série 17HS (dados do 17HS2408). https://baa.vn/document/17hs-series-hanpose-dong-co-buoc-2-pha-lai-6877-catalog1.pdf
4. Wokwi, documentação das peças usadas na simulação: https://docs.wokwi.com/parts/wokwi-a4988 · https://docs.wokwi.com/parts/wokwi-stepper-motor · https://docs.wokwi.com/parts/wokwi-ssd1306
5. Pinagem do Shield CNC v3. https://pcbsync.com/cnc-shield-v3-arduino/

Projetos e vídeos parecidos

6. Masking Tape Plotter: uma máquina que escreve etiquetas num rolo de fita crepe, muito parecida com a de vocês. https://hackaday.io/project/185279-masking-tape-plotter · vídeo: https://www.youtube.com/watch?v=js4_p1S9vIM
7. Motor de passo NEMA com driver A4988 no Arduino: como configurar e testar (em português). https://www.youtube.com/watch?v=A75voFzoYCA
8. Motor de passo com Arduino e driver A4988 (em português). https://www.youtube.com/watch?v=melvk_zZV58
9. Drum Plotter (plotter de tambor). https://www.youtube.com/watch?v=-MjHv8Y4w_8
10. NEMA17 com correia GT2. https://www.youtube.com/watch?v=IHan0WPXAI4
11. Plotter de caneta com GRBL e Arduino. https://www.youtube.com/watch?v=UBH5JA_AIvk

Programas

12. Wokwi (simulador): https://wokwi.com
13. PlatformIO (extensão do VS Code): https://platformio.org
14. GRBL (firmware): https://github.com/gnea/grbl
15. Inkscape (desenho, pode gerar G-code com extensões): https://inkscape.org

APÊNDICE A

O código completo da simulação

O arquivo `sketch.ino` inteiro, com os números de linha usados ao longo da Parte 4. Os outros dois arquivos da simulação, `diagram.json` (o circuito) e `libraries.txt` (as bibliotecas), estão junto com os arquivos do projeto.

SKETCH.INO

```
/*
  Simulacao da Impressora de Etiquetas CNC (Wokwi)

  - Motor do TAMBOR (gira a fita)   -> driver no eixo X do Shield CNC
  - Motor da CANETA (eixo linear)   -> driver no eixo Y do Shield CNC
  - Servo SG90 sobe/desce a caneta
  - Tela OLED faz o papel da FITA: quando a caneta esta abaixada,
    o ponto onde ela esta fica aceso na tela

  Na tela:  horizontal = tambor girando (comprimento da etiqueta)
            vertical   = caneta andando (altura da letra)

  Como usar: ao iniciar, ele escreve "UEFS".
  Depois, digite uma palavra no Serial Monitor e aperte Enter.
  Letras aceitas: A B C D E F G H I L M N O P R S T U V e espaco.

  No Arduino de verdade: e so apagar as linhas da tela (tela.xxx)
  que o resto continua funcionando igual.
*/

#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Servo.h>

// ---------- Pinos (mesmos do Shield CNC v3) ----------
const int TAMBOR_STEP = 2;   // eixo X do shield
const int TAMBOR_DIR  = 5;
const int CANETA_STEP = 3;   // eixo Y do shield
const int CANETA_DIR  = 6;
const int ENABLE_PIN  = 8;   // LOW = drivers ligados
const int SERVO_PIN   = 11;

// ---------- Ajustes ----------
const int CANETA_CIMA  = 90;   // angulo do servo com a caneta levantada
const int CANETA_BAIXO = 30;   // angulo do servo com a caneta encostando na fita

const int ESCALA = 3;          // quantos passos (pixels) vale 1 quadradinho da letra
const int ESPACO_LETRA = 6;    // cada letra ocupa 6 quadradinhos (4 de letra + 2 de espaco)
const int MAX_LETRAS = 7;      // cabe na tela de 128 pixels
const int BASE = 42;           // linha de baixo das letras na tela
const int TEMPO_PASSO_US = 1500;

// ---------- Objetos ----------
Adafruit_SSD1306 tela(128, 64, &Wire, -1);
Servo servo;

// ---------- Posicao atual da maquina ----------
int posTambor = 0;        // em passos (= pixel horizontal na tela)
```

```ino
int posCaneta = BASE;     // em passos (= pixel vertical na tela)
bool canetaAbaixada = false;
int inicioLetra = 0;      // onde comeca a letra que esta sendo escrita

void setup() {
  pinMode(TAMBOR_STEP, OUTPUT);
  pinMode(TAMBOR_DIR, OUTPUT);
  pinMode(CANETA_STEP, OUTPUT);
  pinMode(CANETA_DIR, OUTPUT);
  pinMode(ENABLE_PIN, OUTPUT);
  digitalWrite(ENABLE_PIN, LOW);   // liga os dois drivers

  servo.attach(SERVO_PIN);
  servo.write(CANETA_CIMA);

  Serial.begin(9600);

  tela.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  tela.clearDisplay();
  tela.display();

  Serial.println("Impressora de etiquetas pronta.");
  escreverTexto("UEFS");
  Serial.println("Digite uma palavra e aperte Enter:");
}

void loop() {
  if (Serial.available() > 0) {
    String texto = Serial.readStringUntil('\n');
    texto.trim();
    texto.toUpperCase();

    if (texto.length() > 0) {
      tela.clearDisplay();   // "fita nova"
      tela.display();
      escreverTexto(texto);
      Serial.println("Digite outra palavra:");
    }
  }
}

// ================== TEXTO ==================

void escreverTexto(String texto) {
  if (texto.length() > MAX_LETRAS) {
    Serial.println("Texto grande demais, vou escrever so as 7 primeiras letras.");
  }

  Serial.print("Escrevendo: ");
  Serial.println(texto);

  int quantidade = texto.length();
  for (int i = 0; i < quantidade && i < MAX_LETRAS; i++) {
    inicioLetra = i * ESPACO_LETRA * ESCALA;
    desenharLetra(texto.charAt(i));
  }

  // terminou: levanta a caneta e volta pro comeco
  levantarCaneta();
  irPara(0, BASE);
  Serial.println("Pronto!");
}
```

```
112
113 // Cada letra e desenhada numa grade de 4 de largura por 6 de altura.
114 // (0,0) = canto de baixo a esquerda, (4,6) = canto de cima a direita.
115 void desenharLetra(char letra) {
116   switch (letra) {
117     case 'A':
118       linha(0, 0, 0, 4); linha(0, 4, 2, 6); linha(2, 6, 4, 4); linha(4, 4, 4, 0);
119       linha(0, 3, 4, 3);
120       break;
121     case 'B':
122       linha(0, 0, 0, 6); linha(0, 6, 3, 6); linha(3, 6, 4, 5); linha(4, 5, 4, 4);
123       linha(4, 4, 3, 3); linha(3, 3, 0, 3);
124       linha(3, 3, 4, 2); linha(4, 2, 4, 1); linha(4, 1, 3, 0); linha(3, 0, 0, 0);
125       break;
126     case 'C':
127       linha(4, 6, 0, 6); linha(0, 6, 0, 0); linha(0, 0, 4, 0);
128       break;
129     case 'D':
130       linha(0, 0, 0, 6); linha(0, 6, 2, 6); linha(2, 6, 4, 4); linha(4, 4, 4, 2);
131       linha(4, 2, 2, 0); linha(2, 0, 0, 0);
132       break;
133     case 'E':
134       linha(4, 6, 0, 6); linha(0, 6, 0, 0); linha(0, 0, 4, 0);
135       linha(0, 3, 3, 3);
136       break;
137     case 'F':
138       linha(4, 6, 0, 6); linha(0, 6, 0, 0);
139       linha(0, 3, 3, 3);
140       break;
141     case 'G':
142       linha(4, 6, 0, 6); linha(0, 6, 0, 0); linha(0, 0, 4, 0); linha(4, 0, 4, 3);
143       linha(4, 3, 2, 3);
144       break;
145     case 'H':
146       linha(0, 0, 0, 6);
147       linha(4, 0, 4, 6);
148       linha(0, 3, 4, 3);
149       break;
150     case 'I':
151       linha(1, 6, 3, 6);
152       linha(2, 6, 2, 0);
153       linha(1, 0, 3, 0);
154       break;
155     case 'L':
156       linha(0, 6, 0, 0); linha(0, 0, 4, 0);
157       break;
158     case 'M':
159       linha(0, 0, 0, 6); linha(0, 6, 2, 3); linha(2, 3, 4, 6); linha(4, 6, 4, 0);
160       break;
161     case 'N':
162       linha(0, 0, 0, 6); linha(0, 6, 4, 0); linha(4, 0, 4, 6);
163       break;
164     case 'O':
165       linha(0, 0, 0, 6); linha(0, 6, 4, 6); linha(4, 6, 4, 0); linha(4, 0, 0, 0);
166       break;
167     case 'P':
168       linha(0, 0, 0, 6); linha(0, 6, 4, 6); linha(4, 6, 4, 3); linha(4, 3, 0, 3);
169       break;
170     case 'R':
171       linha(0, 0, 0, 6); linha(0, 6, 4, 6); linha(4, 6, 4, 3); linha(4, 3, 0, 3);
172       linha(2, 3, 4, 0);
173       break;
```

```
    case 'S':
      linha(4, 6, 0, 6); linha(0, 6, 0, 3); linha(0, 3, 4, 3); linha(4, 3, 4, 0);
      linha(4, 0, 0, 0);
      break;
    case 'T':
      linha(0, 6, 4, 6);
      linha(2, 6, 2, 0);
      break;
    case 'U':
      linha(0, 6, 0, 0); linha(0, 0, 4, 0); linha(4, 0, 4, 6);
      break;
    case 'V':
      linha(0, 6, 2, 0); linha(2, 0, 4, 6);
      break;
    case ' ':
      break;   // espaco: nao desenha nada
    default:
      Serial.print("Letra nao suportada: ");
      Serial.println(letra);
      break;
  }
}

// Desenha uma reta entre dois pontos da grade da letra
void linha(int x1, int y1, int x2, int y2) {
  // converte da grade da letra para passos dos motores
  int inicioX = inicioLetra + x1 * ESCALA;
  int inicioY = BASE - y1 * ESCALA;
  int fimX = inicioLetra + x2 * ESCALA;
  int fimY = BASE - y2 * ESCALA;

  // se a caneta nao esta no comeco da linha, levanta e vai ate la
  if (posTambor != inicioX || posCaneta != inicioY || !canetaAbaixada) {
    levantarCaneta();
    irPara(inicioX, inicioY);
    abaixarCaneta();
  }

  irPara(fimX, fimY);
}

// ================= MOVIMENTO =================

// Move os dois motores juntos ate o ponto (xAlvo, yAlvo) em linha reta
void irPara(int xAlvo, int yAlvo) {
  int xInicio = posTambor;
  int yInicio = posCaneta;
  int dx = xAlvo - xInicio;
  int dy = yAlvo - yInicio;

  // quantos pontinhos tem no caminho (o maior dos dois deslocamentos)
  int pontos = max(abs(dx), abs(dy));

  for (int i = 1; i <= pontos; i++) {
    int x = xInicio + round((float)dx * i / pontos);
    int y = yInicio + round((float)dy * i / pontos);

    while (posTambor < x) passoTambor(1);
    while (posTambor > x) passoTambor(-1);
    while (posCaneta < y) passoCaneta(1);
    while (posCaneta > y) passoCaneta(-1);

```

```
    if (canetaAbaixada) {
      marcarFita();
    }
  }
}

void passoTambor(int direcao) {
  if (direcao > 0) {
    digitalWrite(TAMBOR_DIR, HIGH);
  } else {
    digitalWrite(TAMBOR_DIR, LOW);
  }
  pulso(TAMBOR_STEP);
  posTambor = posTambor + direcao;
}

void passoCaneta(int direcao) {
  if (direcao > 0) {
    digitalWrite(CANETA_DIR, HIGH);
  } else {
    digitalWrite(CANETA_DIR, LOW);
  }
  pulso(CANETA_STEP);
  posCaneta = posCaneta + direcao;
}

// Um pulso no STEP = o motor anda 1 passo
void pulso(int pinoStep) {
  digitalWrite(pinoStep, HIGH);
  delayMicroseconds(TEMPO_PASSO_US);
  digitalWrite(pinoStep, LOW);
  delayMicroseconds(TEMPO_PASSO_US);
}

// ================== CANETA ==================

void abaixarCaneta() {
  if (!canetaAbaixada) {
    servo.write(CANETA_BAIXO);
    delay(200);
    canetaAbaixada = true;
    marcarFita();
  }
}

void levantarCaneta() {
  if (canetaAbaixada) {
    servo.write(CANETA_CIMA);
    delay(200);
    canetaAbaixada = false;
  }
}

// So existe na simulacao: acende o ponto onde a caneta esta
void marcarFita() {
  tela.drawPixel(posTambor, posCaneta, SSD1306_WHITE);
  tela.display();
}
```